



ITAHARI
INTERNATIONAL
COLLEGE



Module Code & Module Title

CC6012NT: Data and Web Development

40% Individual Coursework

Submission: Milestone 1

Academic Semester: Autumn Semester 2025

Student Name: Aashish Khadka

London Met ID: 23049144

College ID: np05cp4a230004

Assignment Due Date: 24th December 2025

Assignment Submission Date: 24th December 2025

Submitted To: Mrs. Delish Khadka

Word Count: 5902

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

40% Individual Coursework - First Milestone

 Islington College, Nepal

Document Details

Submission ID

trn:oid::3618:125098482

Submission Date

Dec 24, 2025, 12:13 PM GMT+5:45

Download Date

Dec 24, 2025, 12:16 PM GMT+5:45

File Name

23049144 Aashish khadka M1.pdf

File Size

2.8 MB

53 Pages

4,197 Words

25,039 Characters



28% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Small Matches (less than 8 words)

Match Groups

-  **82 Not Cited or Quoted 24%**
Matches with neither in-text citation nor quotation marks
-  **12 Missing Quotations 4%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 28%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 82 Not Cited or Quoted 24%**
Matches with neither in-text citation nor quotation marks
- 12 Missing Quotations 4%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 0% Internet sources
- 0% Publications
- 28% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	
	Islington College,Nepal on 2025-12-24	8%
2	Submitted works	
	Islington College,Nepal on 2025-12-24	4%
3	Submitted works	
	Islington College,Nepal on 2025-12-24	2%
4	Submitted works	
	Islington College,Nepal on 2025-12-24	1%
5	Submitted works	
	Islington College,Nepal on 2025-12-23	1%
6	Submitted works	
	Islington College,Nepal on 2025-12-24	<1%
7	Submitted works	
	Asia Pacific University College of Technology and Innovation (UCTI) on 2020-03-03	<1%
8	Submitted works	
	Islington College,Nepal on 2025-12-24	<1%
9	Submitted works	
	Islington College,Nepal on 2025-12-24	<1%
10	Submitted works	
	Islington College,Nepal on 2025-12-24	<1%

11	Submitted works	Monash University on 2023-10-26	<1%
12	Submitted works	Islington College,Nepal on 2025-12-24	<1%
13	Submitted works	Islington College,Nepal on 2025-12-24	<1%
14	Submitted works	Islington College,Nepal on 2025-12-24	<1%
15	Submitted works	Sydney Institute of Technology and Commerce on 2024-04-28	<1%
16	Submitted works	Islington College,Nepal on 2025-12-23	<1%
17	Submitted works	Islington College,Nepal on 2025-12-23	<1%
18	Submitted works	Islington College,Nepal on 2025-12-24	<1%
19	Submitted works	Islington College,Nepal on 2025-12-24	<1%
20	Submitted works	ESoft Metro Campus, Sri Lanka on 2025-12-18	<1%
21	Submitted works	Islington College,Nepal on 2025-12-23	<1%
22	Submitted works	Islington College,Nepal on 2025-12-24	<1%
23	Submitted works	Islington College,Nepal on 2025-12-24	<1%
24	Submitted works	The University of Wolverhampton on 2023-05-31	<1%

25	Submitted works	Islington College,Nepal on 2025-12-24	<1%
26	Submitted works	Islington College,Nepal on 2025-12-19	<1%
27	Submitted works	Islington College,Nepal on 2025-12-23	<1%
28	Submitted works	Islington College,Nepal on 2025-12-24	<1%
29	Submitted works	Islington College,Nepal on 2025-12-24	<1%
30	Submitted works	Islington College,Nepal on 2025-12-24	<1%
31	Submitted works	Islington College,Nepal on 2025-12-24	<1%
32	Submitted works	Islington College,Nepal on 2025-12-24	<1%
33	Submitted works	islingtoncollege on 2025-06-08	<1%
34	Submitted works	Islington College,Nepal on 2025-12-23	<1%
35	Submitted works	Islington College,Nepal on 2025-12-24	<1%
36	Submitted works	Islington College,Nepal on 2025-12-24	<1%
37	Submitted works	Islington College,Nepal on 2025-12-24	<1%
38	Submitted works	Islington College,Nepal on 2025-12-24	<1%

39	Submitted works	Islington College,Nepal on 2025-12-24	<1%
40	Submitted works	Islington College,Nepal on 2025-12-24	<1%
41	Submitted works	Islington College,Nepal on 2025-12-24	<1%
42	Submitted works	Islington College,Nepal on 2025-12-24	<1%
43	Submitted works	Islington College,Nepal on 2025-12-24	<1%

Table of Contents

1	Introduction.....	1
2	Tools and Technologies Used	2
3	Initial ERD	3
3.1	Entity and Attributes	3
3.2	Identification of Entities and Attributes.....	3
3.3	Relationships Between Entities (Initial ERD Cardinality)	6
3.4	Initial ERD Diagram Description	7
4	Normalization	9
4.1	(UNF) Un-Normalized Form	9
4.2	1NF (First Normal Form).....	10
4.2.1	User-1	10
4.2.2	Booking-1	11
4.2.3	Price_Policy-1	11
4.2.4	Ticket-1	11
4.2.5	Payment-1	11
4.3	Final 1NF Tables.....	12
4.4	2NF (Second Normal Form).....	13
4.4.1	User-2.....	13
4.4.2	Movie-2.....	14
4.4.3	Theater-2	14
4.4.4	Hall-2	15
4.4.5	Showtime-2.....	16
4.4.6	Booking-2	16
4.4.7	Price_Policy-2.....	17
4.4.8	Ticket-2	18
4.4.9	Payment-2	19
4.5	Final 2NF Tables.....	20
4.6	3NF (Third Normal Form).....	21
4.6.1	Transitive Dependency Check for User-2.....	21
4.6.2	Transitive Dependency Check for Movie-2.....	21

4.6.3	Transitive Dependency Check for Theater-2	22
4.6.4	Transitive Dependency Check for Hall-2	22
4.6.5	Transitive Dependency Check for Showtime-2	22
4.6.6	Transitive Dependency Check for Booking-2	23
4.6.7	Transitive Dependency Check for Price_Policy-2	23
4.6.8	Transitive Dependency Check for Ticket-2	24
4.6.9	Transitive Dependency Check for Payment-2	24
4.7	Final 3NF Tables	25
5	Final ERD	26
6	SQL Implementation and Query Results	27
7	Creating Tables	30
7.1.1	Creating Users table	30
7.1.2	Creating Movie table	30
7.1.3	Creating Theater table	31
7.1.4	Creating Hall table	31
7.1.5	Creating Showtime table	32
7.1.6	Creating Booking table	32
7.1.7	Creating Price_policy table	33
7.1.8	Creating table Ticket	34
7.1.9	Creating table Payment	34
7.2	Insert Queries	36
7.2.1	Inserting values into Users	36
7.2.2	Selecting table Users	36
7.2.3	Inserting values into Movie	37
7.2.4	Selecting table Movie	37
7.2.5	Inserting values into Theater	38
7.2.6	Selecting table Theater	38
7.2.7	Inserting values into Hall	39
7.2.8	Selecting table Hall	40
7.2.9	Inserting values into Showtime	41
7.2.10	Selecting table Showtime	41

7.2.11	Inserting values into Price_policy.....	42
7.2.12	Selecting table Price_Policy	43
7.2.13	Inserting values into Booking	43
7.2.14	Selecting table Booking	44
7.2.15	Inserting values into Ticket.....	45
7.2.16	Inserting values into Payment.....	46
7.2.17	Selecting table Payment.....	46
7.2.18	Query1.....	48
7.2.19	Query 2.....	50
7.2.20	Query 3.....	51
8	Conclusion	53
9	References.....	54
10	Appendix.....	55

Table Of Figures

Figure 1: Relationship of User and Booking	6
Figure 2: Relationship of Booking and Ticket.....	6
Figure 3: Initial ERD	8
Figure 4: Final ERD.....	26
Figure 5: Docker Container	27
Figure 6: Connection created using sysdba	28
Figure 7: Created new connection with username and password	29
Figure 8:Creating Users table	30
Figure 9:Creating Movie table.....	30
Figure 10: Creating Theater table	31
Figure 11: Creating Hall table.....	31
Figure 12: Creating Showtime table	32
Figure 13: Creating Booking table.....	32
Figure 14: Creating Price_policy table	33
Figure 15: Creating ticket table.....	34
Figure 16: Creating payment table.....	34
Figure 17: Sucessfully created all tables.....	35
Figure 18: Inserting into Users	36
Figure 19: Selecting table users	36
Figure 20: Output of selecting user table.....	37
Figure 21: Inserting into Movie	37
Figure 22:Selecting table movie	37
Figure 23: Output of Selecting table Movie	38
Figure 24: Inserting into Theater	38
Figure 25:Selecting table Theater	39
Figure 26:Output of Selecting table Theater.....	39
Figure 27: Inserting into Hall.....	39
Figure 28: Selecting table Hall	40
Figure 29: Output of Selecting table Hall	40
Figure 30: Inserting into Showtime	41
Figure 31: Selecting table Showtime	41
Figure 32: Output of Selecting table Showtime.....	42
Figure 33: Inserting into Price_policy	42
Figure 34:Selecting table Price_policy	43
Figure 35: Outpu of Selecting table Price_policy.....	43
Figure 36: Inserting into Booking.....	44
Figure 37: Selecting table Boking.....	44
Figure 38: Output of Selecting table Booking	44
Figure 39: Inserting into Ticket.....	45

Figure 40: Selecting table Ticket	45
Figure 41: Output of Selecting table Ticket.....	46
Figure 42: Inserting into Payment.....	46
Figure 43: Selecting table Payment	47
Figure 44: Output of Selecting table Payment.....	47
Figure 45: Creating table of User Ticket.....	48
Figure 46: Assigning p_start_ to User Ticket	48
Figure 47: Assigning USer_id in user ticket	49
Figure 48: Output of asssigning user ticket	49
Figure 49: Creating TheaterCityHall Moive table	50
Figure 50: Assigning Hall id	50
Figure 51: Output of above query	51
Figure 52: Creating MovieTheaterCityHall Occupancy performer table	51
Figure 53: Assigning movie_ID.....	52
Figure 54: Output of above query	52

Tables Of Table

Table 1: Initial identification of USER entity and its attributes.....	3
Table 2: Initial identification of BOOKING entity and its attributes	4
Table 3: Initial identification of TICKET entity and its attributes.....	5

1 Introduction

KumariCinemas aims to establish a modern cinema chain in Nepal by implementing a centralized and automated movie ticket management system. The organization is currently experiencing problems of hall management, scheduling of shows, pricing of tickets, and customer experience. The unstructured nature of the data handling will bring about redundancy, inconsistency and inefficiencies in operations. The given coursework is dedicated to the creation of a strong database system of KumariCinemas through the adequate normalization and the usage of the Entity Relationship Diagrams (ERDs). The database reaches the Third Normal Form (3NF) of normalization and implemented with the Oracle SQL developer to guarantee the database integrity, high scalability, and effective management of tickets.

2 Tools and Technologies Used

1. Oracle SQL Developer Data Modeler

The final normalised schema was implemented on Oracle Database 19c as the target DBMS. It offers good relational support mechanisms which include primary keys, foreign keys, NOTNULL, and datatype constraints. Oracle 19c can be used in transaction systems such as ticketing since it is a system that guarantees the integrity of data and the ability to maintain the usual multi-user operations of the system. In this coursework, Oracle made sure that table relationships (e.g. ticket-to-showtime and showtime-to-hall) were implemented properly. The Oracle is also compatible with the trusted SQL execution and validation with SQL plus and SQL developer to ensure that the database could be tested with insertions and query outputs. (Oracle, 2025)

2. SQL*Plus and Oracle SQL Developer

The Oracle database was connected to through SQLPlus whereby, users were created, privileges assigned, quotas set and the scripts generated when creating the tables used. It gives a simple method of executing scripts and validation of success messages to table creation and constraint creation. The GUI tool of Oracle SQL Developer was utilized to see schema objects, execute queries with formatted results and take screenshot of SQL queries and results to be used in the report. Combined they were able to assist with command-line validation (SQLPlus) and visual evidence (SQL Developer) mandated by the milestone deliverables. (Oracle, 2025)

3 Initial ERD

3.1 Entity and Attributes

In database design, an entity is an object or a concept in the real world that contains data and can be identified. Every entity will have a set of attributes which define its properties. The idea of the initial ERD is to obtain the structure of the system as it is presented in the un-normalised data without dividing the attributes into several refined objects. (Paradigm, 2025)

As per the un-normalised booking data and the original ERD diagram, the KumariCinemas system initial comprises of three primary entities namely: USER, BOOKING and TICKET. All these are the basic booking and payment process prior to the application of the normalization process.

3.2 Identification of Entities and Attributes

The initial ERD is based on the integrated data based on which booking, movie, theater, hall, showtime, ticket, and payment are saved. Attributes in this stage are categorized under larger entities that show how the data is initially prevalent of what normalization entails.

USER

Table 1: Initial identification of USER entity and its attributes

S. N	Attribute Name	Data Type	Description
1	UserID	INT	Unique identifier for each user
2	Username	STRING	Name of the user
3	Address	STRING	Address of the user

Explanation:

The USER entity stores basic information about customers who use the cinema booking system. Each user is uniquely identified using UserID. At the initial stage, only essential user details are captured without further separation.

BOOKING

Table 2: Initial identification of BOOKING entity and its attributes

S.N	Attribute Name	Data Type	Description
1	BookingID	INT	Unique identifier for each booking
2	BookingDateTime	DATETIME	Date and time when booking was made
3	BookingStatus	STRING	Status of the booking
4	MovieID	INT	Identifier of the booked movie
5	MovieTitle	STRING	Title of the movie
6	Duration	INT	Duration of the movie
7	Language	STRING	Language of the movie
8	Genre	STRING	Genre of the movie
9	ReleaseDate	DATE	Movie release date

10	TheaterName	STRING	Name of the theater
11	TheaterCity	STRING	City where theater is located
12	HallNumber	INT	Hall number
13	HallCapacity	INT	Seating capacity of the hall
14	ShowDate	DATE	Date of the show
15	ShowTime	TIME	Time of the show

Explanation:

The BOOKING entity represents a cinema booking made by a user. In the initial ERD, booking information also contains movie, theater, hall, and showtime details. These attributes are grouped together because, in the un-normalised dataset, they are stored within a single booking record.

TICKET

Table 3: Initial identification of TICKET entity and its attributes

S.N	Attribute Name	Data Type	Description
1	TicketID	INT	Unique identifier for each ticket
2	TicketStatus	STRING	Current status of the ticket
3	SeatNumber	STRING	Allocated seat number
4	PaymentID	INT	Identifier of the payment
5	PaymentMethod	STRING	Method used for payment
6	PaymentAmount	FLOAT	Amount paid

7	PaymentStatus	STRING	Status of the payment
8	TransactionRef	STRING	Payment transaction reference
9	PaymentTime	DATETIME	Date and time of payment

Explanation:

The TICKET entity represents the ticket issued after a booking. In the initial ERD, ticket and payment-related attributes are combined because payment details are directly associated with a ticket in the un-normalised structure.

3.3 Relationships Between Entities (Initial ERD Cardinality)

The relationships identified in the initial ERD are based on how the data is connected in the un-normalised structure:

- One USER can make multiple BOOKINGS, but each booking is made by only one user. (One-to-Many: USER 1 — M BOOKING)

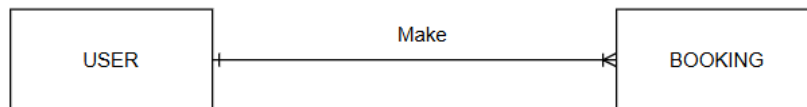


Figure 1: Relationship of User and Booking

- One BOOKING can contain multiple TICKETS, but each ticket belongs to only one booking. (One-to-Many: BOOKING 1 — M TICKET)

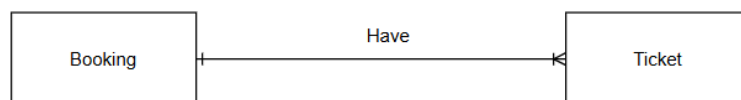


Figure 2: Relationship of Booking and Ticket

These relationships reflect the real-world cinema booking process and match the structure shown in the initial ERD diagram.

3.4 Initial ERD Diagram Description

The Initial ERD is a graphical representation of the three key entities, such as USER, BOOKING, and TICKET, their attributes, and relationships. The diagram indicates that user and ticket booking information are centrally positioned and that movie, hall, showtime, and payment information is embedded in the bookings and tickets. This ERD is used as the beginning point of normalization in subsequent stages where redundancy problems and dependency problems would be solved.

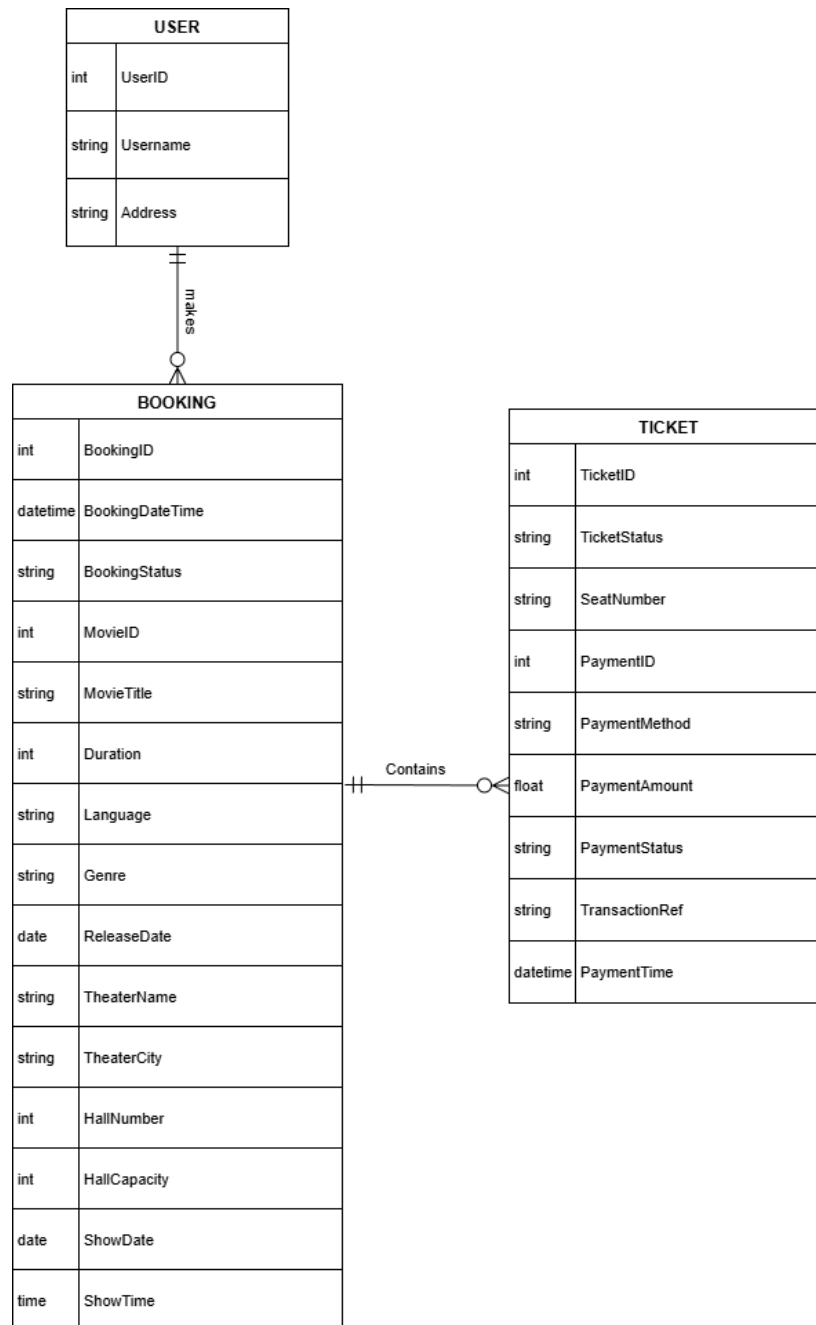


Figure 3: Initial ERD

4 Normalization

The process of organizing data in a database to attain quality and enhance efficiency in storage is known as normalization and the process is referred to as data arrangement. This is formulation of tables according to some principles in order to eliminate dependency and redundancy. Although in normalization the procedure of eliminating duplicated data may include the arrangement of information in a logical and comprehensible way, it must depict the various types of entities, their properties and the connection among them in a proper manner. The goals are non-duplicable and the data integrity. And its needs are:

- i. Normalization eliminates data redundancy so as to conserve storage.
- ii. It averts the cases of insertion, deletion and update anomalies.
- iii. It ensures the data integrity in the database.
- iv. Normalization involves use of primary and foreign key to enhance data integrity.
- v. And it classifies data in a logical manner in order to maximize the performance of queries.

4.1 (UNF) Un-Normalized Form

It is the very normalized form, or even un-normalized form, where information is stored in some sort of database. In UNF, several repeating arrays and groups in a single table appear redundant in one table, and are not very efficient. It also represents a simplistic and quite haphazard form of data used on at least one opening on the process of normalizing to remove repetition in a bid to make it more specific. It is unstructured and unrefined means of storing information. This format allowed them to use the same information several times.

KumariCinema_UNF

User (UserID, Username, Address, {BookingID, BookingDateTime, BookingStatus, MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate, TheaterName, TheaterCity, HallNumber, HallCapacity, ShowDate, ShowTime, {EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, TicketPrice}, {TicketID, TicketStatus, SeatNumber, PaymentID, PaymentMethod, PaymentAmount, PaymentStatus, TransactionRef, PaymentTime}})

Details:

- User: Holds basic information of customers who use the system to book tickets.
- Booking: Stores details of each movie reservation made by a user.
- Movie: Keeps records of movies available for screening.
- Theater: Represents cinema locations where movies are displayed.
- Hall: Describes individual screening halls and their seating capacity.
- Showtime: Records the scheduled date and time of movie screenings.
- TicketPrice: Defines ticket cost based on dates, holidays, and release periods.
- Ticket: Contains information about issued tickets, seats, and status.
- Payment: Stores transaction details related to ticket purchases.

4.2 1NF (First Normal Form)

In 1NF any database table does not contain any lists or arrays; instead of this it contains just simple, indivisible values in each column. In this form, every row must be different and no two rows should be identical. So, every record from this form can be easily identified and it prevents the repetition of data sets. Some key points are listed below;

- Repeating groups were removed by separating Booking, Price Policy, Ticket, and Payment into individual relations.
- Each attribute in every relation now holds a single value (atomic).

Process for 1NF: Converting UNF into 1NF

4.2.1 User-1

User-1: (UserID, Username, Address)

Details:

- UserID is the primary key of User.
- Username and Address are atomic and non-repeating attributes.
- This removes repeated user information from the nested structure.

4.2.2 Booking-1

Booking-1: (BookingID, BookingDateTime, BookingStatus, MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate, TheaterName, TheaterCity, HallNumber, HallCapacity, ShowDate, ShowTime)

Details:

- BookingID is the primary key of Booking.
- BookingDateTime and BookingStatus are atomic.
- Movie, Theater, Hall, and Showtime details are still stored inside Booking at 1NF stage, which creates redundancy but does not break 1NF.

4.2.3 Price_Policy-1

Price_Policy-1: (EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, TicketPrice)

Details:

- This relation stores atomic pricing rule attributes.
- At this stage, the price rule is separated as a repeating group from UNF, but it is not yet linked by identifiers (fixed properly in 2NF).

4.2.4 Ticket-1

Ticket-1: (TicketID, TicketStatus, SeatNumber)

Details:

- TicketID is the primary key.
- TicketStatus and SeatNumber are atomic values.
- Tickets are now stored as individual records instead of a nested set.

4.2.5 Payment-1

Payment-1: (PaymentID, PaymentMethod, PaymentAmount, PaymentStatus, TransactionRef, PaymentTime)

Details:

- PaymentID is the primary key.
- All payment-related attributes are atomic.
- Payment is extracted from the nested repeating group.

4.3 Final 1NF Tables

- User-1 (UserID, Username, Address)
- Booking-1 (BookingID, BookingDateTime, BookingStatus, MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate, TheaterName, TheaterCity, HallNumber, HallCapacity, ShowDate, ShowTime)
- Price_Policy-1 (EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, TicketPrice)
- Ticket-1 (TicketID, TicketStatus, SeatNumber)
- Payment-1 (PaymentID, PaymentMethod, PaymentAmount, PaymentStatus, TransactionRef, PaymentTime)

4.4 2NF (Second Normal Form)

In 2NF the objective is to remove partial dependencies arising when a part of a composite primary key is dependent on some attributes. To this form, the table needs to be in 1NF. The next thing would be that all non key attributes would have to depend on the complete key and not just part of the key. Some of the key points of 2nf are listed below;

- Remove partial dependency by ensuring all non-key attributes depend on the whole primary key.
- Separate entity details that were mixed inside Booking-1 (Movie, Theater, Hall, Showtime).
- Introduce proper identifiers and foreign keys to connect relations.

Process for 2NF (identify Partial Dependencies and resolve using IDs):

4.4.1 User-2

User-2: (UserID, Username, Address)

1. Full Functional Dependency (FFD):

User-2: (UserID, Username, Address)

- FFD exists because UserID uniquely determines Username and Address.
- All non-key attributes depend entirely on UserID.

2. Partial Functional Dependency:

User-2: (UserID, Username, Address)

No partial dependency exists because UserID is a single-attribute primary key.

Composite Key:

User-2:

- No composite primary key is used.

4.4.2 Movie-2

Movie-2: (MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate)

1. Full Functional Dependency (FFD):

Movie-2: (MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate)

- FFD exists because MovieID uniquely determines MovieTitle, Duration, Language, Genre, and ReleaseDate.
- All non-key attributes depend entirely on MovieID.

2. Partial Functional Dependency:

Movie-2:

- No partial dependency exists because MovieID is a single-attribute primary key.

Composite Key:

Movie-2:

- No composite primary key is used.

4.4.3 Theater-2

Theater-2: (TheaterID, TheaterName, TheaterCity)

1. Full Functional Dependency (FFD):

Theater-2: (TheaterID, TheaterName, TheaterCity)

- FFD exists because TheaterID uniquely determines TheaterName and TheaterCity.

2. Partial Functional Dependency:

Theater-2:

- No partial dependency exists because TheaterID is a single-attribute primary key.

Composite Key:**Theater-2:**

- No composite primary key is used.

4.4.4 Hall-2

Hall-2: (HallID, TheaterID*, HallNumber, HallCapacity)

1. Full Functional Dependency (FFD):

Hall-2: (HallID, TheaterID*, HallNumber, HallCapacity)

- FFD exists because HallID uniquely determines TheaterID, HallNumber, and HallCapacity.

2. Partial Functional Dependency:**Hall-2:**

- No partial dependency exists because HallID is a single-attribute primary key.

Composite Key:**Hall-2:**

- No composite primary key is used.

A logical uniqueness constraint may be applied as (TheaterID, HallNumber) to ensure hall numbers are not duplicated within the same theater.

4.4.5 Showtime-2

Showtime-2: (ShowID, HallID*, MovieID*, ShowDate, ShowTime)

1. Full Functional Dependency (FFD):

Showtime-2: (ShowID, HallID*, MovieID*, ShowDate, ShowTime)

- FFD exists because ShowID uniquely determines HallID, MovieID, ShowDate, and ShowTime.

2. Partial Functional Dependency:

Showtime-2:

No partial dependency exists because ShowID is a single-attribute primary key.

Composite Key:

Showtime-2:

- No composite primary key is used.

A logical composite uniqueness constraint can be defined as (HallID, ShowDate, ShowTime) to prevent scheduling conflicts.

4.4.6 Booking-2

Booking-2: (BookingID, UserID*, ShowID*, BookingDateTime, BookingStatus)

1. Full Functional Dependency (FFD):

Booking-2: (BookingID, UserID*, ShowID*, BookingDateTime, BookingStatus)

- FFD exists because BookingID uniquely determines UserID, ShowID, BookingDateTime, and BookingStatus.

2. Partial Functional Dependency:

Booking-2:

- No partial dependency exists because BookingID is a single-attribute primary key.

Composite Key:

Booking-2:

- No composite primary key is used.

The combination (UserID, ShowID, BookingDateTime) can act as a logical uniqueness constraint if required by business rules.

4.4.7 Price_Policy-2

Price_Policy-2: (PolicyID, HallID*, MovieID*, EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, TicketPrice)

1. Full Functional Dependency (FFD):

Price_Policy-2: (PolicyID, HallID*, MovieID*, EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, TicketPrice)

- FFD exists because PolicyID uniquely determines HallID, MovieID, EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, and TicketPrice.

2. Partial Functional Dependency:

Price_Policy-2:

- No partial dependency exists because PolicyID is a single-attribute primary key.

Composite Key:

Price_Policy-2:

- No composite primary key is used.

A logical composite uniqueness constraint can be defined as (HallID, MovieID, EffectiveFromDate) to avoid duplicate overlapping policies.

4.4.8 Ticket-2

Ticket-2: (TicketID, BookingID*, SeatNumber, TicketStatus)

1. Full Functional Dependency (FFD):

Ticket-2: (TicketID, BookingID*, SeatNumber, TicketStatus)

- FFD exists because TicketID uniquely determines BookingID, SeatNumber, and TicketStatus.

2. Partial Functional Dependency:

Ticket-2:

- No partial dependency exists because TicketID is a single-attribute primary key.

Composite Key:

Ticket-2:

- No composite primary key is used.

A logical uniqueness constraint can be defined as (BookingID, SeatNumber) to prevent duplicate seat assignment in the same booking.

4.4.9 Payment-2

Payment-2: (PaymentID, BookingID*, PaymentMethod, PaymentAmount, PaymentStatus, TransactionRef, PaymentTime)

1. Full Functional Dependency (FFD):

Payment-2: (PaymentID, BookingID*, PaymentMethod, PaymentAmount, PaymentStatus, TransactionRef, PaymentTime)

- FFD exists because PaymentID uniquely determines BookingID and all payment attributes.

2. Partial Functional Dependency:

Payment-2:

No partial dependency exists because PaymentID is a single-attribute primary key.

Composite Key:

Payment-2:

- No composite primary key is used.

4.5 Final 2NF Tables

- User-2 (UserID, Username, Address)
- Movie-2 (MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate)
- Theater-2 (TheaterID, TheaterName, TheaterCity)
- Hall-2 (HallID, TheaterID*, HallNumber, HallCapacity)
- Showtime-2 (ShowID, HallID*, MovieID*, ShowDate, ShowTime)
- Booking-2 (BookingID, UserID*, ShowID*, BookingDateTime, BookingStatus)
- Price_Policy-2 (PolicyID, HallID*, MovieID*, EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, TicketPrice)
- Ticket-2 (TicketID, BookingID*, SeatNumber, TicketStatus)
- Payment-2 (PaymentID, BookingID*, PaymentMethod, PaymentAmount, PaymentStatus, TransactionRef, PaymentTime, TicketStatus).

4.6 3NF (Third Normal Form)

In 3NF, it cleans a database by removing non-key attributes that link to other non-key attributes out of a 2NF format. To reach 3NF we must ensure that our table follows both 1NF and 2NF standards first. The table must show how each non-key attribute relates only to the primary key attribute without connection to other non-key attributes. Each table element gains direct access to the primary key thanks to this concept that avoids duplicate data. Some of the key points of 3nf are listed below;

- Remove transitive dependency (non-key attribute depending on another non-key attribute).
- Verify each 2NF relation to ensure no descriptive attribute depends on another descriptive attribute.

4.6.1 Transitive Dependency Check for User-2

User-2: (UserID, Username, Address)

- No transitive dependency exists because Username and Address depend directly on UserID.
- No non-key attribute determines another non-key attribute.

So newly form table is

User-3: (UserID, Username, Address)

4.6.2 Transitive Dependency Check for Movie-2

Movie-2: (MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate)

- No transitive dependency exists because all attributes depend directly on MovieID.
- No attribute such as Genre determines other attributes in the table.

So newly form table is

Movie-3: (MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate)

4.6.3 Transitive Dependency Check for Theater-2

Theater-2: (TheaterID, TheaterName, TheaterCity)

- No transitive dependency exists because TheaterName and TheaterCity depend directly on TheaterID.

So newly form table is

Theater-3: (TheaterID, TheaterName, TheaterCity)

4.6.4 Transitive Dependency Check for Hall-2

Hall-2: (HallID, TheaterID*, HallNumber, HallCapacity)

- No transitive dependency exists because HallNumber, HallCapacity, and TheaterID depend directly on HallID.
- TheaterName and TheaterCity are not stored here, so no dependency chain can occur inside this table.

So, the formed table is

Hall-3: (HallID, TheaterID*, HallNumber, HallCapacity)

4.6.5 Transitive Dependency Check for Showtime-2

Showtime-2: (ShowID, HallID*, MovieID*, ShowDate, ShowTime)

- No transitive dependency exists because ShowDate and ShowTime depend directly on ShowID.
- MovieTitle and Hall details are not stored in this table, preventing transitive chains.

So, the formed table is

Showtime-3: (ShowID, HallID*, MovieID*, ShowDate, ShowTime)

4.6.6 Transitive Dependency Check for Booking-2

Booking-2: (BookingID, UserID*, ShowID*, BookingDateTime, BookingStatus)

- No transitive dependency exists because BookingDateTime and BookingStatus depend directly on BookingID.
- Username is not stored here, so $\text{BookingID} \rightarrow \text{UserID} \rightarrow \text{Username}$ does not occur inside this table.

So, the formed table is

Booking-3: (BookingID, UserID*, ShowID*, BookingDateTime, BookingStatus)

4.6.7 Transitive Dependency Check for Price_Policy-2

Price_Policy-2: (PolicyID, HallID*, MovieID*, EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, TicketPrice)

- No transitive dependency exists because all pricing attributes depend directly on PolicyID.
- HallNumber or MovieTitle are not stored here, so transitive dependency is avoided.

Price_Policy-3: (PolicyID, HallID*, MovieID*, EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, TicketPrice)

4.6.8 Transitive Dependency Check for Ticket-2

Ticket-2: (TicketID, BookingID*, SeatNumber, TicketStatus)

- No transitive dependency exists because SeatNumber and TicketStatus depend directly on TicketID.
- User details are not stored in this table, so no transitive chain occurs.

So, the formed table is

Ticket-3: (TicketID, BookingID*, SeatNumber, TicketStatus)

4.6.9 Transitive Dependency Check for Payment-2

Payment-2: (PaymentID, BookingID*, PaymentMethod, PaymentAmount, PaymentStatus, TransactionRef, PaymentTime)

- No transitive dependency exists because all payment attributes depend directly on PaymentID.
- Booking details are not stored here, preventing transitive dependency.

Payment-3: (PaymentID, BookingID*, PaymentMethod, PaymentAmount, PaymentStatus, TransactionRef, PaymentTime)

4.7 Final 3NF Tables

- User-3: (UserID, Username, Address)
- Movie-3: (MovieID, MovieTitle, Duration, Language, Genre, ReleaseDate)
- Theater-3: (TheaterID, TheaterName, TheaterCity)
- Hall-3: (HallID, TheaterID*, HallNumber, HallCapacity)
- Showtime-3: (ShowID, HallID*, MovieID*, ShowDate, ShowTime)
- Booking-3: (BookingID, UserID*, ShowID*, BookingDateTime, BookingStatus)
- Price_Policy-3: (PolicyID, HallID*, MovieID*, EffectiveFromDate, EffectiveToDate, IsPublicHoliday, IsNewReleaseWeek, TicketPrice)
- Ticket-3: (TicketID, BookingID*, SeatNumber, TicketStatus)
- Payment-3: (PaymentID, BookingID*, PaymentMethod, PaymentAmount, PaymentStatus, TransactionRef, PaymentTime)

5 Final ERD

This ERD models Kumari Cinema's booking workflow in a clean 3NF structure. **USERS** stores customer details. A user creates **BOOKING** records; each tied to one **SHOWTIME**. **SHOWTIME** represents a scheduled screening and connects a **MOVIE** to a specific **HALL** on a given date and time, with a uniqueness rule to prevent double-booking the same hall at the same slot. **HALL** belongs to a **THEATER**, and the theater information is kept separate to avoid repetition. Tickets are stored in **TICKET**, linked to **BOOKING**, with seat uniqueness per booking. **PAYMENT** records the transaction for a booking. **PRICE_POLICY** defines valid ticket prices by hall, movie, and date conditions.

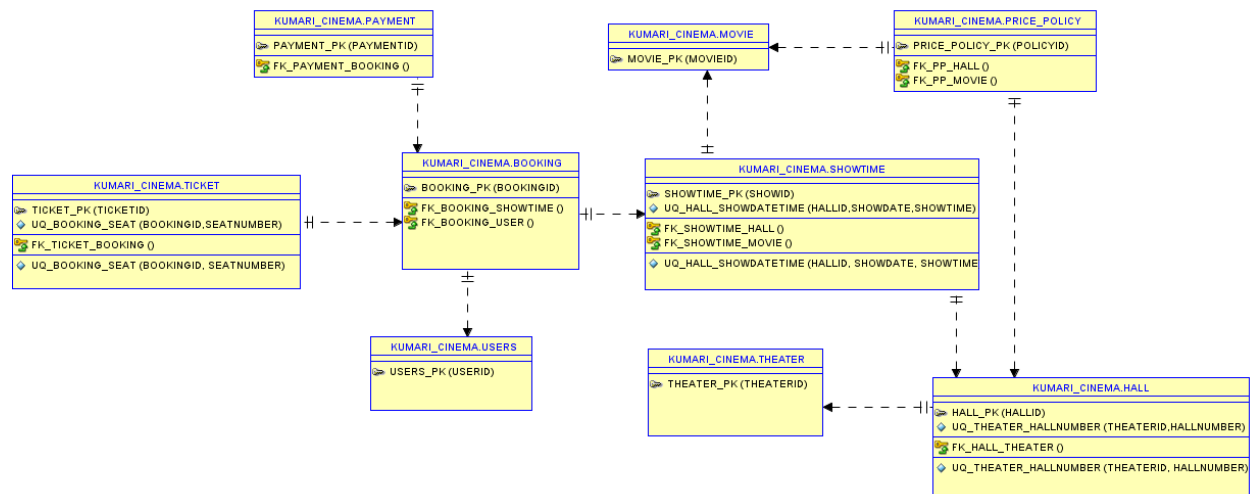


Figure 4: Final ERD

6 SQL Implementation and Query Results

The SQL implementation was performed using Oracle Database 21 xe in a Docker environment to ensure a consistent and controlled database setup. First, a dedicated database user was created and granted appropriate privileges, including connection permission, object creation rights, and unlimited quota on the USER tablespace. After user creation, the DDL script generated by Oracle SQL Developer Data Modeler was executed. The execution produced confirmation outputs such as “Table created” and “Table altered,” indicating that table creation and constraint creation were successful.

Here we have successfully run the docker container and opened in respective terminal

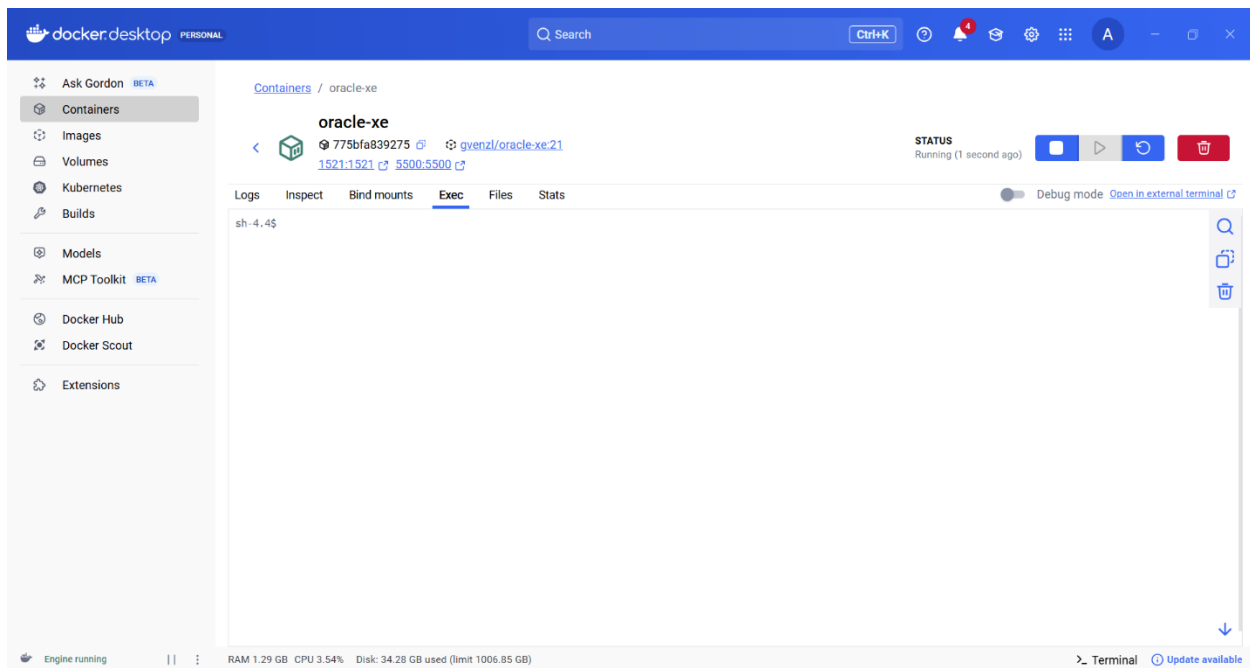


Figure 5: Docker Container

Here we have created a new project and connected with oracle 21 xe where role is SYSDBA and service name is XEPDB1

New / Select Database Connection

Connection Name: Project

Database Type: Oracle

User Info: Proxy User

Authentication Type: Default

Username: sys

Password:

Role: SYSDBA

Save Password: ☒

Connection Type: Basic

Details: Advanced

Hostname: localhost

Port: 1521

☐ SID: xe

☒ Service name: xepdb1

Status : Success

Buttons: Help, Save, Clear, Test, Connect, Cancel

Figure 6: Connection created using sysdba

Here after created the project we have created a new user name KUMARI_CINEMA and connected and tested to oracle sucessfully

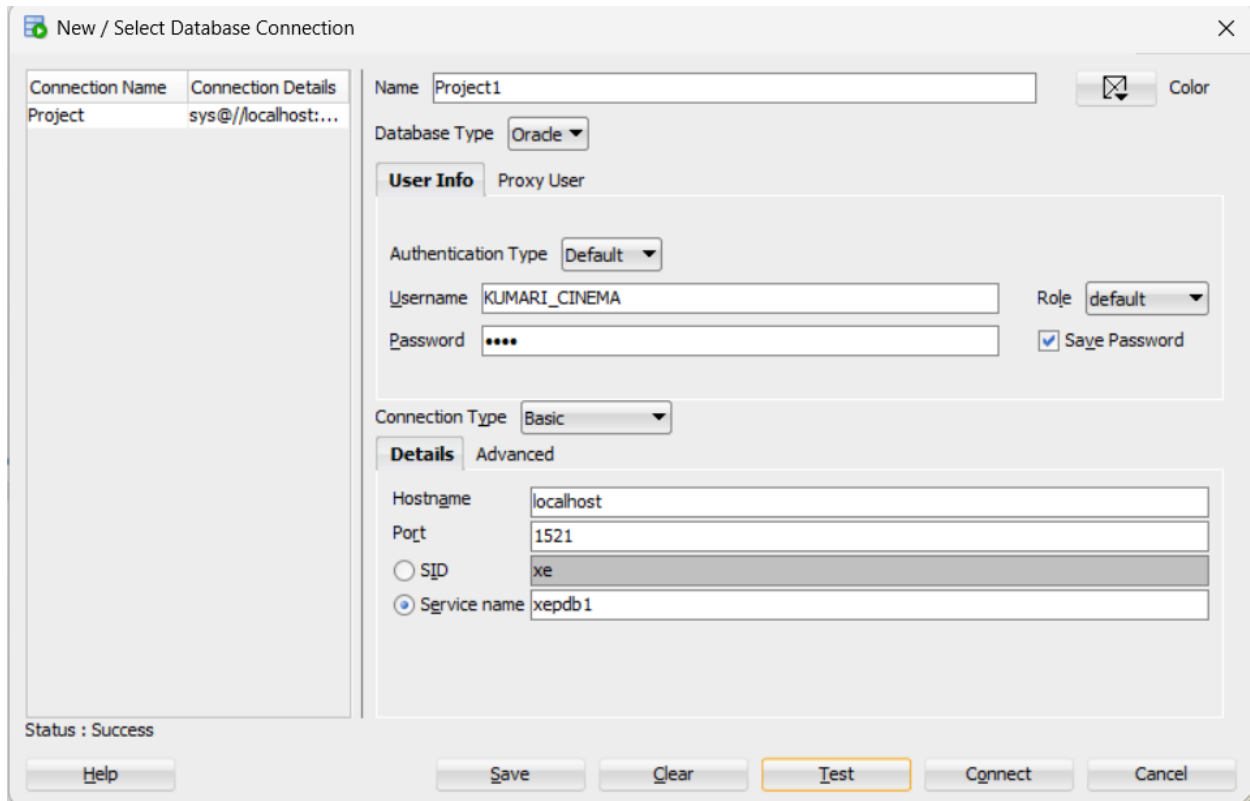


Figure 7: Created new connection with username and password

7 Creating Tables

7.1.1 Creating Users table

```
CREATE TABLE USERS (  
    UserID    NUMBER(10)    PRIMARY KEY,  
    Username  VARCHAR2(80)  NOT NULL,  
    Address   VARCHAR2(80)  NOT NULL  
);
```

Figure 8: Creating Users table

7.1.2 Creating Movie table

```
CREATE TABLE MOVIE (  
    MovieID    NUMBER(10)    PRIMARY KEY,  
    MovieTitle  VARCHAR2(150) NOT NULL,  
    Duration    NUMBER(5),  
    Language    VARCHAR2(50)  NOT NULL,  
    Genre       VARCHAR2(50)  NOT NULL,  
    ReleaseDate DATE  
);
```

Figure 9: Creating Movie table

7.1.3 Creating Theater table

```
CREATE TABLE THEATER (  
    TheaterID    NUMBER(10)    PRIMARY KEY,  
    TheaterName  VARCHAR2(120) NOT NULL,  
    TheaterCity  VARCHAR2(80)  NOT NULL  
);
```

Figure 10: Creating Theater table

7.1.4 Creating Hall table

```
CREATE TABLE HALL (  
    HallID       NUMBER(10)    PRIMARY KEY,  
    TheaterID    NUMBER(10)    NOT NULL,  
    HallNumber   NUMBER(5)     NOT NULL,  
    HallCapacity NUMBER(5)     NOT NULL,  
    CONSTRAINT FK_HALL_THEATER  
        FOREIGN KEY (TheaterID) REFERENCES THEATER(TheaterID),  
    CONSTRAINT CK_HALL_CAPACITY  
        CHECK (HallCapacity > 0),  
    CONSTRAINT UQ_THEATER_HALLNUMBER  
        UNIQUE (TheaterID, HallNumber)  
);
```

Figure 11: Creating Hall table

7.1.5 Creating Showtime table

```
CREATE TABLE SHOWTIME (  
    ShowID      NUMBER(10)  PRIMARY KEY,  
    HallID      NUMBER(10)  NOT NULL,  
    MovieID     NUMBER(10)  NOT NULL,  
    ShowDate    DATE        NOT NULL,  
    ShowTime    VARCHAR2(8) NOT NULL,  
    CONSTRAINT FK_SHOWTIME_HALL  
        FOREIGN KEY (HallID) REFERENCES HALL(HallID),  
    CONSTRAINT FK_SHOWTIME_MOVIE  
        FOREIGN KEY (MovieID) REFERENCES MOVIE(MovieID),  
    CONSTRAINT UQ_HALL_SHOWDATETIME  
        UNIQUE (HallID, ShowDate, ShowTime)  
);
```

Figure 12: Creating Showtime table

7.1.6 Creating Booking table

```
CREATE TABLE BOOKING (  
    BookingID    NUMBER(10)  PRIMARY KEY,  
    UserID       NUMBER(10)  NOT NULL,  
    ShowID       NUMBER(10)  NOT NULL,  
    BookingDateTime TIMESTAMP NOT NULL,  
    BookingStatus VARCHAR2(20) NOT NULL,  
    CONSTRAINT FK_BOOKING_USER  
        FOREIGN KEY (UserID) REFERENCES USERS(UserID),  
    CONSTRAINT FK_BOOKING_SHOWTIME  
        FOREIGN KEY (ShowID) REFERENCES SHOWTIME(ShowID),  
    CONSTRAINT CK_BOOKING_STATUS  
        CHECK (BookingStatus IN ('BOOKED', 'CONFIRMED', 'CANCELLED', 'EXPIRED'))  
);
```

Figure 13: Creating Booking table

7.1.7 Creating Price_policy table

```
CREATE TABLE PRICE_POLICY (  
  PolicyID          NUMBER(10)    PRIMARY KEY,  
  HallID            NUMBER(10)    NOT NULL,  
  MovieID           NUMBER(10)    NOT NULL,  
  EffectiveFromDate DATE          NOT NULL,  
  EffectiveToDate   DATE,  
  IsPublicHoliday   CHAR(1)       NOT NULL,  
  IsNewReleaseWeek  CHAR(1)       NOT NULL,  
  TicketPrice       NUMBER(10,2)  NOT NULL,  
  CONSTRAINT FK_PP_HALL  
    FOREIGN KEY (HallID) REFERENCES HALL(HallID),  
  CONSTRAINT FK_PP_MOVIE  
    FOREIGN KEY (MovieID) REFERENCES MOVIE(MovieID),  
  CONSTRAINT CK_PP_FLAGS  
    CHECK (IsPublicHoliday IN ('Y','N') AND IsNewReleaseWeek IN ('Y','N')),  
  CONSTRAINT CK_PP_PRICE  
    CHECK (TicketPrice > 0)  
);
```

Figure 14: Creating Price_policy table

7.1.8 Creating table Ticket

```
CREATE TABLE TICKET (  
    TicketID      NUMBER(10)    PRIMARY KEY,  
    BookingID     NUMBER(10)    NOT NULL,  
    SeatNumber    VARCHAR2(10)  NOT NULL,  
    TicketStatus  VARCHAR2(20)  NOT NULL,  
    CONSTRAINT FK_TICKET_BOOKING  
        FOREIGN KEY (BookingID) REFERENCES BOOKING(BookingID),  
    CONSTRAINT CK_TICKET_STATUS  
        CHECK (TicketStatus IN ('RESERVED', 'PAID', 'CANCELLED', 'REFUNDED')),  
    CONSTRAINT UQ_BOOKING_SEAT  
        UNIQUE (BookingID, SeatNumber)  
);
```

Figure 15: Creating ticket table

7.1.9 Creating table Payment

```
CREATE TABLE PAYMENT (  
    PaymentID     NUMBER(10)    PRIMARY KEY,  
    BookingID     NUMBER(10)    NOT NULL,  
    PaymentMethod VARCHAR2(20)  NOT NULL,  
    PaymentAmount NUMBER(10,2)  NOT NULL,  
    PaymentStatus VARCHAR2(20)  NOT NULL,  
    TransactionRef VARCHAR2(40),  
    PaymentTime   TIMESTAMP,  
    CONSTRAINT FK_PAYMENT_BOOKING  
        FOREIGN KEY (BookingID) REFERENCES BOOKING(BookingID),  
    CONSTRAINT CK_PAYMENT_METHOD  
        CHECK (PaymentMethod IN ('CASH', 'CARD', 'ESEWA', 'STRIPE')),  
    CONSTRAINT CK_PAYMENT_STATUS  
        CHECK (PaymentStatus IN ('PAID', 'FAILED', 'PENDING', 'REFUNDED')),  
    CONSTRAINT CK_PAYMENT_AMOUNT  
        CHECK (PaymentAmount >= 0)  
);
```

Figure 16: Creating payment table

Output of created tables sucessfully

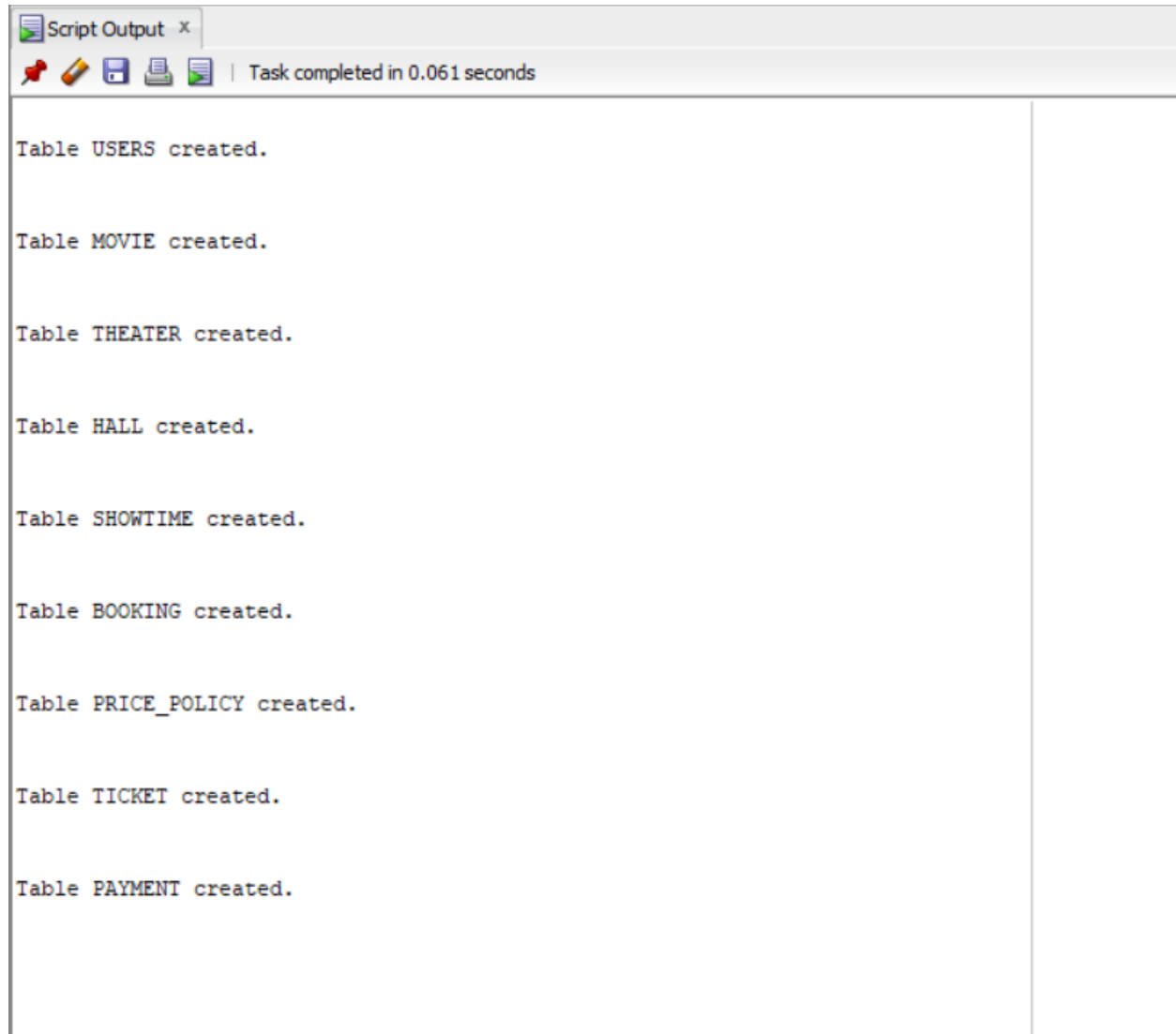


Figure 17: Sucessfully created all tables

7.2 Insert Queries

7.2.1 Inserting values into Users

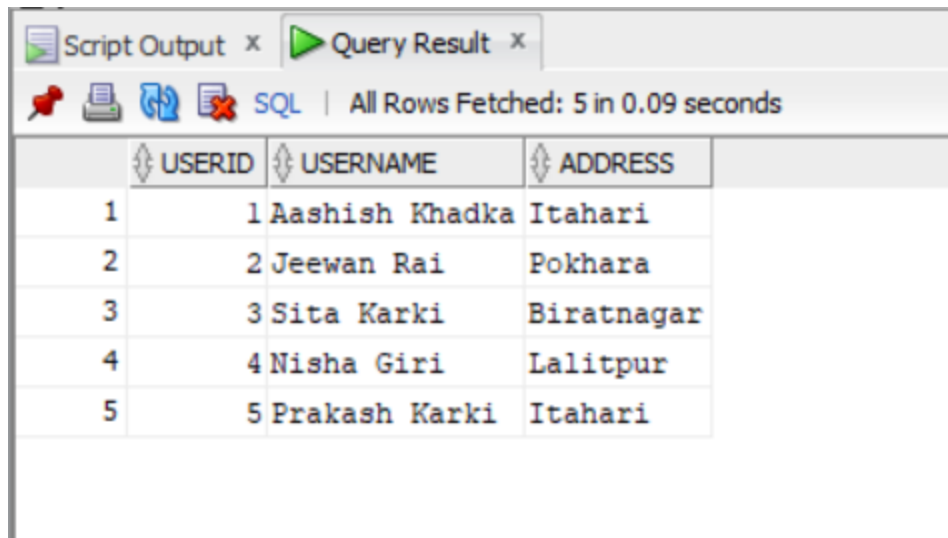
```
INSERT INTO USERS VALUES (1, 'Aashish Khadka', 'Itahari');  
INSERT INTO USERS VALUES (2, 'Jeewan Rai', 'Pokhara');  
INSERT INTO USERS VALUES (3, 'Sita Karki', 'Biratnagar');  
INSERT INTO USERS VALUES (4, 'Nisha Giri', 'Lalitpur');  
INSERT INTO USERS VALUES (5, 'Prakash Karki', 'Itahari');
```

Figure 18: Inserting into Users

7.2.2 Selecting table Users

```
-- select queries --  
  
SELECT * FROM USERS ORDER BY UserID;
```

Figure 19: Selecting table users



	USERID	USERNAME	ADDRESS
1	1	Aashish Khadka	Itahari
2	2	Jeewan Rai	Pokhara
3	3	Sita Karki	Biratnagar
4	4	Nisha Giri	Lalitpur
5	5	Prakash Karki	Itahari

Figure 20: Output of selecting user table

7.2.3 Inserting values into Movie

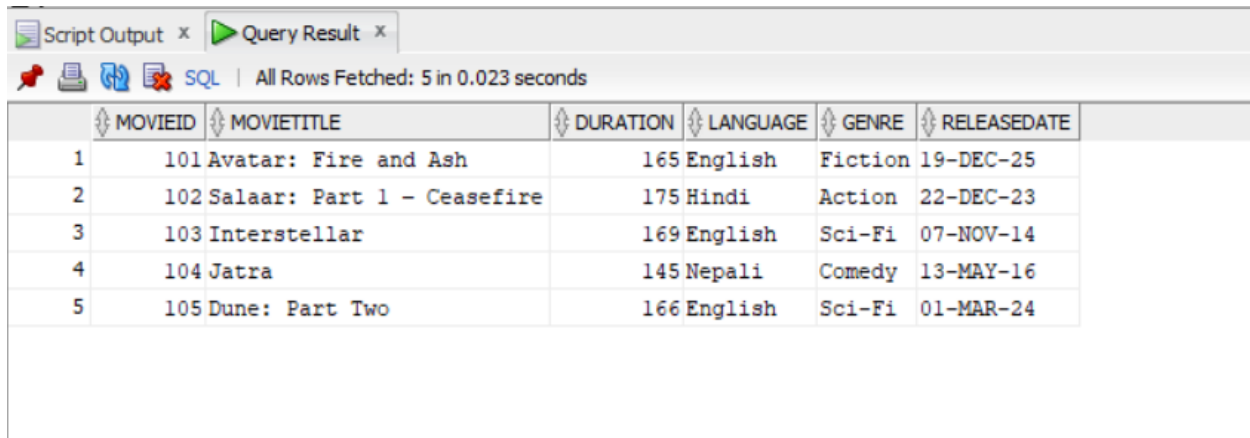
```
INSERT INTO MOVIE VALUES (101,'Avatar: Fire and Ash',165,'English','Fiction', DATE '2025-12-19');
INSERT INTO MOVIE VALUES (102,'Salaar: Part 1 - Ceasefire',175,'Hindi','Action', DATE '2023-12-22');
INSERT INTO MOVIE VALUES (103,'Interstellar',169,'English','Sci-Fi', DATE '2014-11-07');
INSERT INTO MOVIE VALUES (104,'Jatra',145,'Nepali','Comedy', DATE '2016-05-13');
INSERT INTO MOVIE VALUES (105,'Dune: Part Two',166,'English','Sci-Fi', DATE '2024-03-01');
```

Figure 21: Inserting into Movie

7.2.4 Selecting table Movie

```
SELECT * FROM MOVIE ORDER BY MovieID;
```

Figure 22: Selecting table movie



Script Output x Query Result x

SQL | All Rows Fetched: 5 in 0.023 seconds

	MOVIEID	MOVIEITITLE	DURATION	LANGUAGE	GENRE	RELEASEDATE
1	101	Avatar: Fire and Ash	165	English	Fiction	19-DEC-25
2	102	Salaar: Part 1 - Ceasefire	175	Hindi	Action	22-DEC-23
3	103	Interstellar	169	English	Sci-Fi	07-NOV-14
4	104	Jatra	145	Nepali	Comedy	13-MAY-16
5	105	Dune: Part Two	166	English	Sci-Fi	01-MAR-24

Figure 23: Output of Selecting table Movie

7.2.5 Inserting values into Theater

```

INSERT INTO THEATER VALUES (201, 'Kumari Cinemas', 'Kathmandu');
INSERT INTO THEATER VALUES (202, 'Pokhara Cineplex', 'Pokhara');
INSERT INTO THEATER VALUES (203, 'Labim Mall Cinema', 'Lalitpur');
INSERT INTO THEATER VALUES (204, 'Biratnagar Cineplex', 'Biratnagar');
INSERT INTO THEATER VALUES (205, 'Itahari City Cinema', 'Itahari');

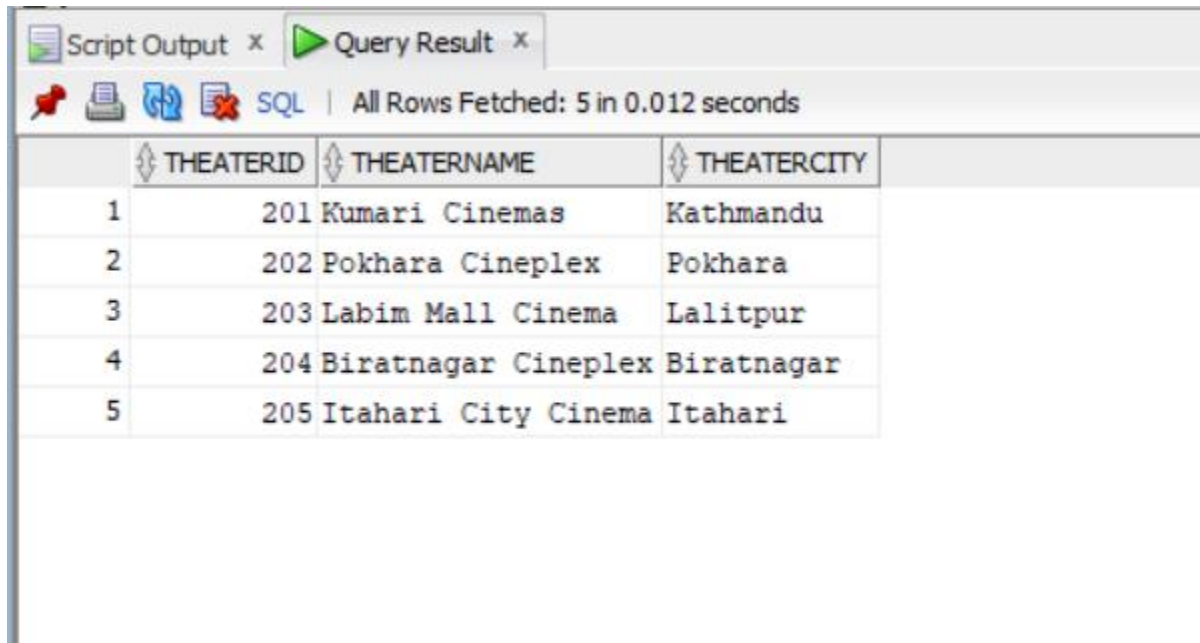
```

Figure 24: Inserting into Theater

7.2.6 Selecting table Theater

```
SELECT * FROM THEATER ORDER BY TheaterID;
```

Figure 25: Selecting table Theater



	THEATERID	THEATERNAME	THEATERCITY
1	201	Kumari Cinemas	Kathmandu
2	202	Pokhara Cineplex	Pokhara
3	203	Labim Mall Cinema	Lalitpur
4	204	Biratnagar Cineplex	Biratnagar
5	205	Itahari City Cinema	Itahari

Figure 26: Output of Selecting table Theater

7.2.7 Inserting values into Hall

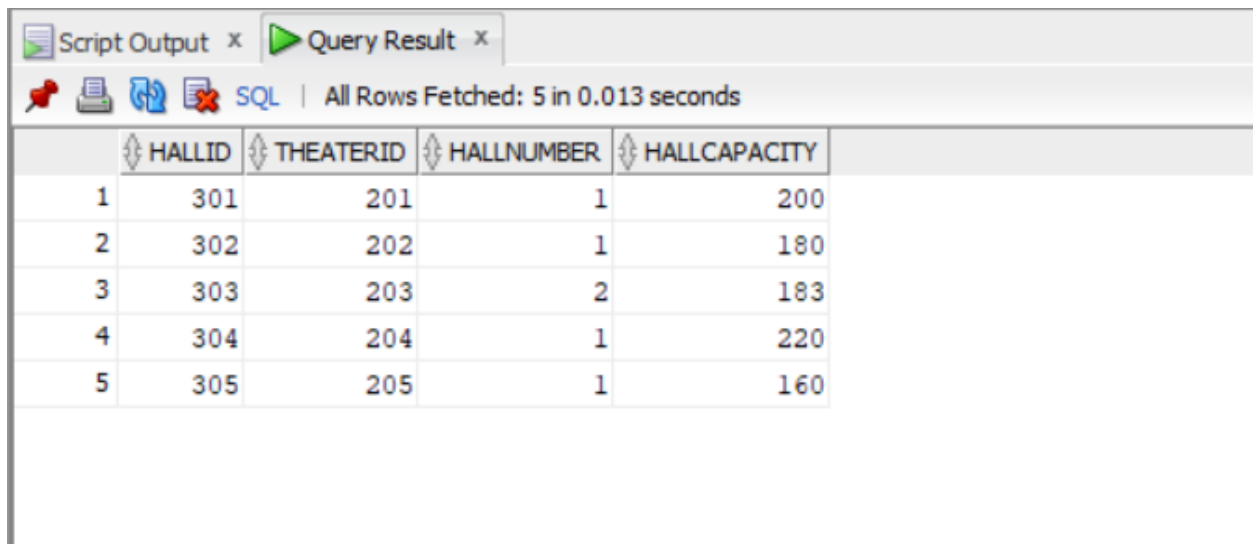
```
INSERT INTO HALL VALUES (301,201,1,200);
INSERT INTO HALL VALUES (302,202,1,180);
INSERT INTO HALL VALUES (303,203,2,183);
INSERT INTO HALL VALUES (304,204,1,220);
INSERT INTO HALL VALUES (305,205,1,160);
```

Figure 27: Inserting into Hall

7.2.8 Selecting table Hall

```
SELECT * FROM HALL ORDER BY HallID;
```

Figure 28: Selecting table Hall



	HALLID	THEATERID	HALLNUMBER	HALLCAPACITY
1	301	201	1	200
2	302	202	1	180
3	303	203	2	183
4	304	204	1	220
5	305	205	1	160

Figure 29: Output of Selecting table Hall

7.2.9 Inserting values into Showtime

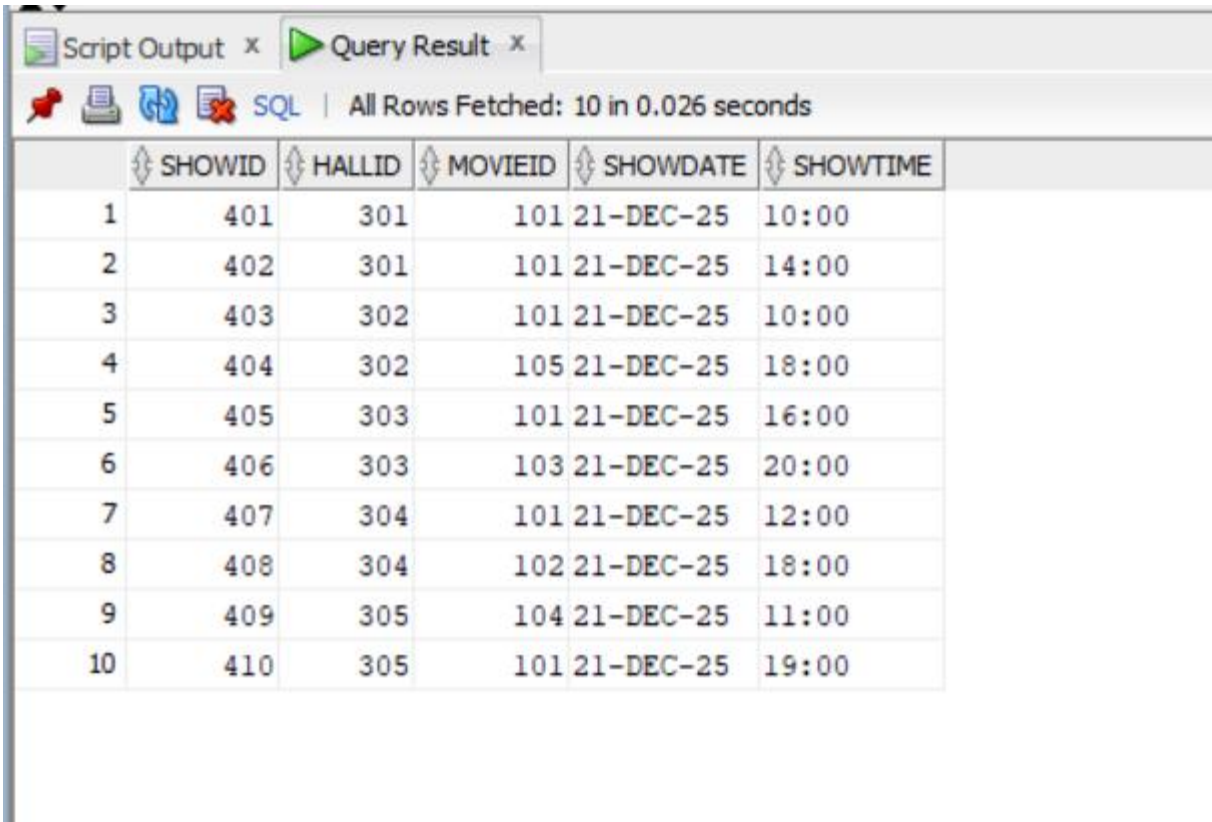
```
INSERT INTO SHOWTIME VALUES (401,301,101,DATE '2025-12-21','10:00');  
INSERT INTO SHOWTIME VALUES (402,301,101,DATE '2025-12-21','14:00');  
INSERT INTO SHOWTIME VALUES (403,302,101,DATE '2025-12-21','10:00');  
INSERT INTO SHOWTIME VALUES (404,302,105,DATE '2025-12-21','18:00');  
INSERT INTO SHOWTIME VALUES (405,303,101,DATE '2025-12-21','16:00');  
INSERT INTO SHOWTIME VALUES (406,303,103,DATE '2025-12-21','20:00');  
INSERT INTO SHOWTIME VALUES (407,304,101,DATE '2025-12-21','12:00');  
INSERT INTO SHOWTIME VALUES (408,304,102,DATE '2025-12-21','18:00');  
INSERT INTO SHOWTIME VALUES (409,305,104,DATE '2025-12-21','11:00');  
INSERT INTO SHOWTIME VALUES (410,305,101,DATE '2025-12-21','19:00');
```

Figure 30: Inserting into Showtime

7.2.10 Selecting table Showtime

```
SELECT * FROM SHOWTIME ORDER BY ShowID;
```

Figure 31: Selecting table Showtime



Script Output x Query Result x

SQL | All Rows Fetched: 10 in 0.026 seconds

	SHOWID	HALLID	MOVIEID	SHOWDATE	SHOWTIME
1	401	301	101	21-DEC-25	10:00
2	402	301	101	21-DEC-25	14:00
3	403	302	101	21-DEC-25	10:00
4	404	302	105	21-DEC-25	18:00
5	405	303	101	21-DEC-25	16:00
6	406	303	103	21-DEC-25	20:00
7	407	304	101	21-DEC-25	12:00
8	408	304	102	21-DEC-25	18:00
9	409	305	104	21-DEC-25	11:00
10	410	305	101	21-DEC-25	19:00

Figure 32: Output of Selecting table Showtime

7.2.11 Inserting values into Price_policy

```

INSERT INTO PRICE_POLICY VALUES (501,301,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',450.00);
INSERT INTO PRICE_POLICY VALUES (502,301,101,DATE '2025-12-20',DATE '2025-12-22','Y','Y',550.00);
INSERT INTO PRICE_POLICY VALUES (503,302,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',420.00);
INSERT INTO PRICE_POLICY VALUES (504,302,105,DATE '2025-12-01',DATE '2025-12-31','N','N',480.00);
INSERT INTO PRICE_POLICY VALUES (505,303,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',430.00);
INSERT INTO PRICE_POLICY VALUES (506,303,103,DATE '2025-12-01',DATE '2025-12-31','N','N',400.00);
INSERT INTO PRICE_POLICY VALUES (507,304,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',460.00);
INSERT INTO PRICE_POLICY VALUES (508,304,102,DATE '2025-12-01',DATE '2025-12-31','N','N',440.00);
INSERT INTO PRICE_POLICY VALUES (509,305,104,DATE '2025-12-01',DATE '2025-12-31','N','N',350.00);
INSERT INTO PRICE_POLICY VALUES (510,305,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',410.00);

```

Figure 33: Inserting into Price_policy

7.2.12 Selecting table Price_Policy

```
SELECT * FROM PRICE_POLICY ORDER BY PolicyID;
```

Figure 34: Selecting table Price_policy

Script Output x

Query Result x

 | All Rows Fetched: 10 in 0.019 seconds

	POLICYID	HALLID	MOVIEID	EFFECTIVEFROMDATE	EFFECTIVETODATE	ISPUBLICHOLIDAY	ISNEWRELEASEWEEK	TICKETPRICE
1	501	301	101	01-DEC-25	31-DEC-25	N	Y	450
2	502	301	101	20-DEC-25	22-DEC-25	Y	Y	550
3	503	302	101	01-DEC-25	31-DEC-25	N	Y	420
4	504	302	105	01-DEC-25	31-DEC-25	N	N	480
5	505	303	101	01-DEC-25	31-DEC-25	N	Y	430
6	506	303	103	01-DEC-25	31-DEC-25	N	N	400
7	507	304	101	01-DEC-25	31-DEC-25	N	Y	460
8	508	304	102	01-DEC-25	31-DEC-25	N	N	440
9	509	305	104	01-DEC-25	31-DEC-25	N	N	350
10	510	305	101	01-DEC-25	31-DEC-25	N	Y	410

Figure 35: Outpu of Selecting table Price_policy

7.2.13 Inserting values into Booking

```

INSERT INTO BOOKING VALUES (601,1,401,TO_TIMESTAMP('2025-12-20 10:15:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
INSERT INTO BOOKING VALUES (602,2,401,TO_TIMESTAMP('2025-12-20 11:10:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
INSERT INTO BOOKING VALUES (603,3,402,TO_TIMESTAMP('2025-12-20 12:05:00','YYYY-MM-DD HH24:MI:SS'),'BOOKED');
INSERT INTO BOOKING VALUES (604,4,403,TO_TIMESTAMP('2025-12-20 12:40:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
INSERT INTO BOOKING VALUES (605,5,404,TO_TIMESTAMP('2025-12-20 13:20:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
INSERT INTO BOOKING VALUES (606,1,405,TO_TIMESTAMP('2025-12-20 14:10:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
INSERT INTO BOOKING VALUES (607,2,407,TO_TIMESTAMP('2025-12-20 15:30:00','YYYY-MM-DD HH24:MI:SS'),'CANCELLED');
INSERT INTO BOOKING VALUES (608,3,408,TO_TIMESTAMP('2025-12-20 16:45:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
INSERT INTO BOOKING VALUES (609,4,410,TO_TIMESTAMP('2025-12-20 17:10:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
INSERT INTO BOOKING VALUES (610,5,406,TO_TIMESTAMP('2025-12-20 18:05:00','YYYY-MM-DD HH24:MI:SS'),'BOOKED');

```

Figure 36: Inserting into Booking

7.2.14 Selecting table Booking

```
SELECT * FROM BOOKING ORDER BY BookingID;
```

Figure 37: Selecting table Boking

Script Output x Query Result x

 All Rows Fetched: 10 in 0.019 seconds

	BOOKINGID	USERID	SHOWID	BOOKINGDATETIME	BOOKINGSTATUS
1	601	1	401	20-DEC-25 10.15.00.000000000 AM	CONFIRMED
2	602	2	401	20-DEC-25 11.10.00.000000000 AM	CONFIRMED
3	603	3	402	20-DEC-25 12.05.00.000000000 PM	BOOKED
4	604	4	403	20-DEC-25 12.40.00.000000000 PM	CONFIRMED
5	605	5	404	20-DEC-25 01.20.00.000000000 PM	CONFIRMED
6	606	1	405	20-DEC-25 02.10.00.000000000 PM	CONFIRMED
7	607	2	407	20-DEC-25 03.30.00.000000000 PM	CANCELLED
8	608	3	408	20-DEC-25 04.45.00.000000000 PM	CONFIRMED
9	609	4	410	20-DEC-25 05.10.00.000000000 PM	CONFIRMED
10	610	5	406	20-DEC-25 06.05.00.000000000 PM	BOOKED

Figure 38: Output of Selecting table Booking

7.2.15 Inserting values into Ticket

```
INSERT INTO TICKET VALUES (701,601,'A1','PAID');  
INSERT INTO TICKET VALUES (702,601,'A2','PAID');  
INSERT INTO TICKET VALUES (703,602,'B1','PAID');  
INSERT INTO TICKET VALUES (704,603,'C1','RESERVED');  
INSERT INTO TICKET VALUES (705,603,'C2','RESERVED');  
INSERT INTO TICKET VALUES (706,604,'D5','PAID');  
INSERT INTO TICKET VALUES (707,605,'E3','PAID');  
INSERT INTO TICKET VALUES (708,605,'E4','PAID');  
INSERT INTO TICKET VALUES (709,606,'F1','PAID');  
INSERT INTO TICKET VALUES (710,607,'G2','CANCELLED');  
INSERT INTO TICKET VALUES (711,608,'H1','PAID');  
INSERT INTO TICKET VALUES (712,609,'J9','PAID');
```

Figure 39: Inserting into Ticket

```
SELECT * FROM TICKET ORDER BY TicketID;
```

Figure 40: Selecting table Ticket

Script Output x Query Result x

 | All Rows Fetched: 12 in 0.02 seconds

	TICKETID	BOOKINGID	SEATNUMBER	TICKETSTATUS
1	701	601 A1	PAID	
2	702	601 A2	PAID	
3	703	602 B1	PAID	
4	704	603 C1	RESERVED	
5	705	603 C2	RESERVED	
6	706	604 D5	PAID	
7	707	605 E3	PAID	
8	708	605 E4	PAID	
9	709	606 F1	PAID	
10	710	607 G2	CANCELLED	
11	711	608 H1	PAID	
12	712	609 J9	PAID	

Figure 41: Output of Selecting table Ticket

7.2.16 Inserting values into Payment

```
INSERT INTO PAYMENT VALUES (801, 601, 'ESEWA', 900.00, 'PAID', 'ESW-601-01', TO_TIMESTAMP('2025-12-20 10:20:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO PAYMENT VALUES (802, 602, 'CARD', 420.00, 'PAID', 'CRD-602-01', TO_TIMESTAMP('2025-12-20 11:15:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO PAYMENT VALUES (803, 603, 'ESEWA', 0.00, 'PENDING', 'ESW-603-01', NULL);
INSERT INTO PAYMENT VALUES (804, 604, 'CASH', 420.00, 'PAID', 'CSH-604-01', TO_TIMESTAMP('2025-12-20 12:45:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO PAYMENT VALUES (805, 605, 'STRIPE', 960.00, 'PAID', 'STP-605-01', TO_TIMESTAMP('2025-12-20 13:25:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO PAYMENT VALUES (806, 606, 'CARD', 430.00, 'PAID', 'CRD-606-01', TO_TIMESTAMP('2025-12-20 14:15:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO PAYMENT VALUES (807, 607, 'CARD', 0.00, 'REFUNDED', 'CRD-607-01', TO_TIMESTAMP('2025-12-20 15:35:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO PAYMENT VALUES (808, 608, 'ESEWA', 440.00, 'PAID', 'ESW-608-01', TO_TIMESTAMP('2025-12-20 16:50:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO PAYMENT VALUES (809, 609, 'CARD', 410.00, 'PAID', 'CRD-609-01', TO_TIMESTAMP('2025-12-20 17:15:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO PAYMENT VALUES (810, 610, 'CASH', 0.00, 'FAILED', 'CSH-610-01', TO_TIMESTAMP('2025-12-20 18:10:00', 'YYYY-MM-DD HH24:MI:SS'));
```

Figure 42: Inserting into Payment

7.2.17 Selecting table Payment

```
SELECT * FROM PAYMENT ORDER BY PaymentID;
```

Figure 43: Selecting table Payment

Script Output x Query Result x

All Rows Fetched: 10 in 0.028 seconds

	PAYMENTID	BOOKINGID	PAYMENTMETHOD	PAYMENTAMOUNT	PAYMENTSTATUS	TRANSACTIONREF	PAYMENTTIME
1	801	601	ESEWA	900	PAID	ESW-601-01	20-DEC-25 10.20.00.000000000 AM
2	802	602	CARD	420	PAID	CRD-602-01	20-DEC-25 11.15.00.000000000 AM
3	803	603	ESEWA	0	PENDING	ESW-603-01	(null)
4	804	604	CASH	420	PAID	CSH-604-01	20-DEC-25 12.45.00.000000000 PM
5	805	605	STRIPE	960	PAID	STP-605-01	20-DEC-25 01.25.00.000000000 PM
6	806	606	CARD	430	PAID	CRD-606-01	20-DEC-25 02.15.00.000000000 PM
7	807	607	CARD	0	REFUNDED	CRD-607-01	20-DEC-25 03.35.00.000000000 PM
8	808	608	ESEWA	440	PAID	ESW-608-01	20-DEC-25 04.50.00.000000000 PM
9	809	609	CARD	410	PAID	CRD-609-01	20-DEC-25 05.15.00.000000000 PM
10	810	610	CASH	0	FAILED	CSH-610-01	20-DEC-25 06.10.00.000000000 PM

Figure 44: Output of Selecting table Payment

7.2.18 Query1

```

/* Q1: User Ticket (6 months) */
SELECT
  u.UserID, u.Username, u.Address,
  b.BookingID, b.BookingDateTime, b.BookingStatus,
  t.TicketID, t.SeatNumber, t.TicketStatus,
  p.PaymentMethod, p.PaymentAmount, p.PaymentStatus, p.TransactionRef, p.PaymentTime
FROM USERS u
JOIN BOOKING b ON b.UserID = u.UserID
JOIN TICKET t ON t.BookingID = b.BookingID
JOIN PAYMENT p ON p.BookingID = b.BookingID
WHERE u.UserID = :p_user_id
      AND b.BookingDateTime >= TO_TIMESTAMP(:p_start_ts, 'YYYY-MM-DD HH24:MI:SS')
      AND b.BookingDateTime < ADD_MONTHS(TO_TIMESTAMP(:p_start_ts, 'YYYY-MM-DD HH24:MI:SS'), 6)
      AND p.PaymentStatus = 'PAID'
ORDER BY b.BookingDateTime, t.TicketID;

```

Figure 45: Creating table of User Ticket

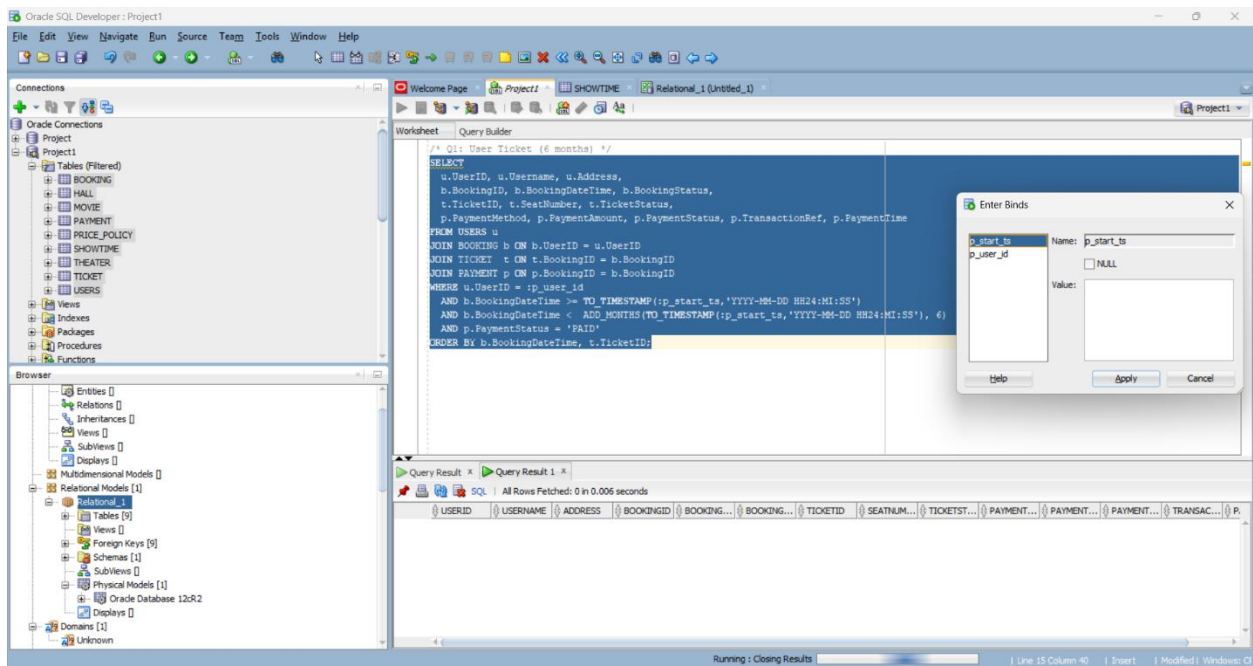


Figure 46: Assigning *p_start_to* User Ticket

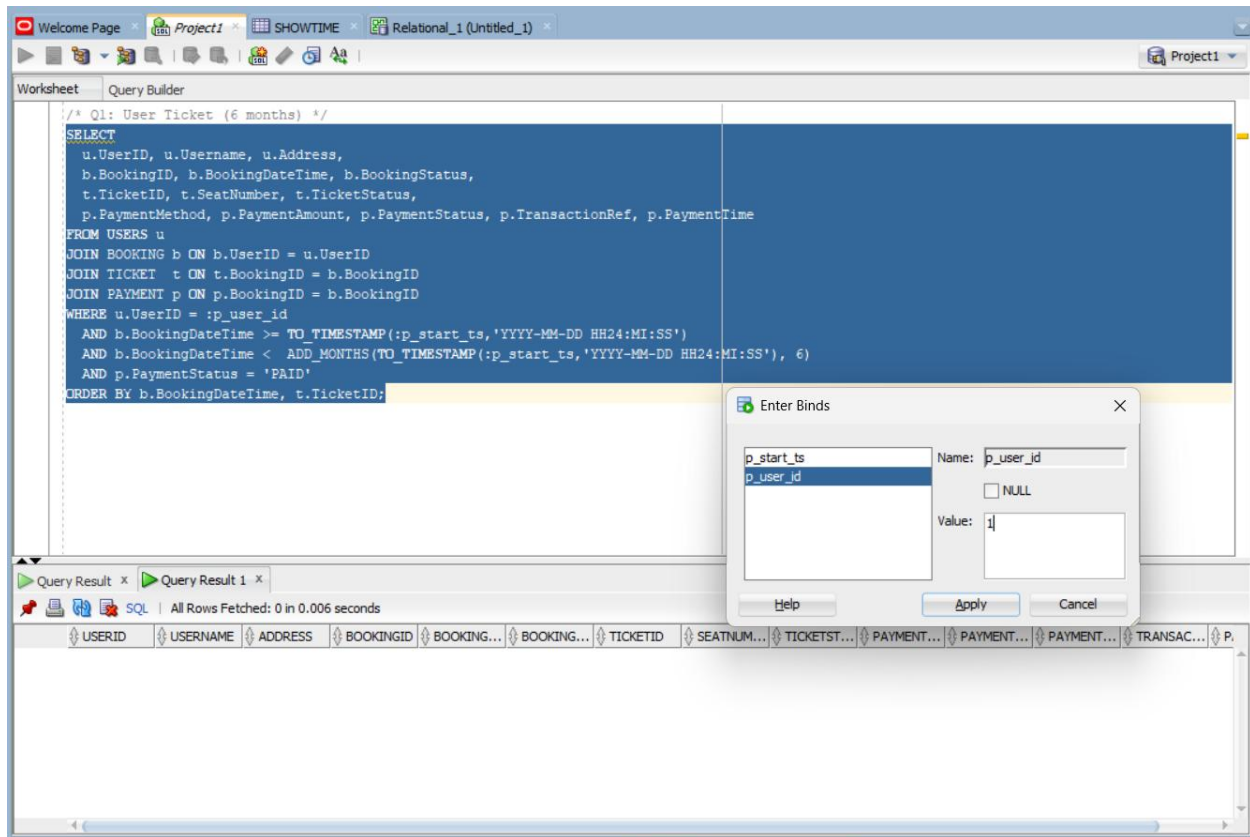


Figure 47: Assigning USer_id in user ticket

Query Result x Query Result 1 x

 All Rows Fetched: 3 in 0.032 seconds

	USERID	USERNAME	ADDRESS	BOOKINGID	BOOKINGDATETIME	BOOKINGSTATUS	TICKETID	SEATNUMBER	TICKETSTATUS	PAYMENTMETHOD	PAYMENTAMOUNT	PAYMENTSTATUS	TRANSACTIONREF	PAYMENTTIME
1	1	Aashish Khadka Itahari		60120-DEC-25	10.15.00.0000000000	AM CONFIRMED	701A1	PAID	ESEWA		900 PAID		ESW-601-01	20-DEC-25 10.20.00.0000
2	1	Aashish Khadka Itahari		60120-DEC-25	10.15.00.0000000000	AM CONFIRMED	702A2	PAID	ESEWA		900 PAID		ESW-601-01	20-DEC-25 10.20.00.0000
3	1	Aashish Khadka Itahari		60620-DEC-25	02.10.00.0000000000	PH CONFIRMED	709F1	PAID	CARD		430 PAID		CRD-606-01	20-DEC-25 02.15.00.0000

Figure 48: Output of assigning user ticket

7.2.19 Query 2

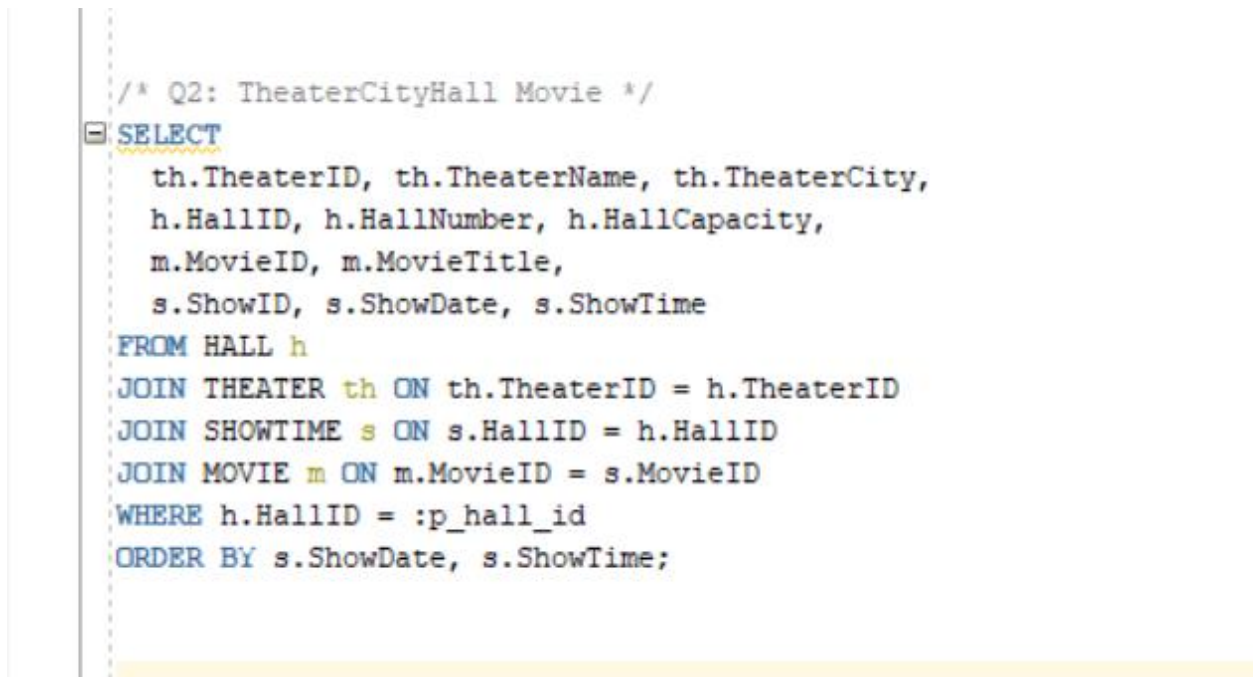


Figure 49: Creating TheaterCityHall Movie table

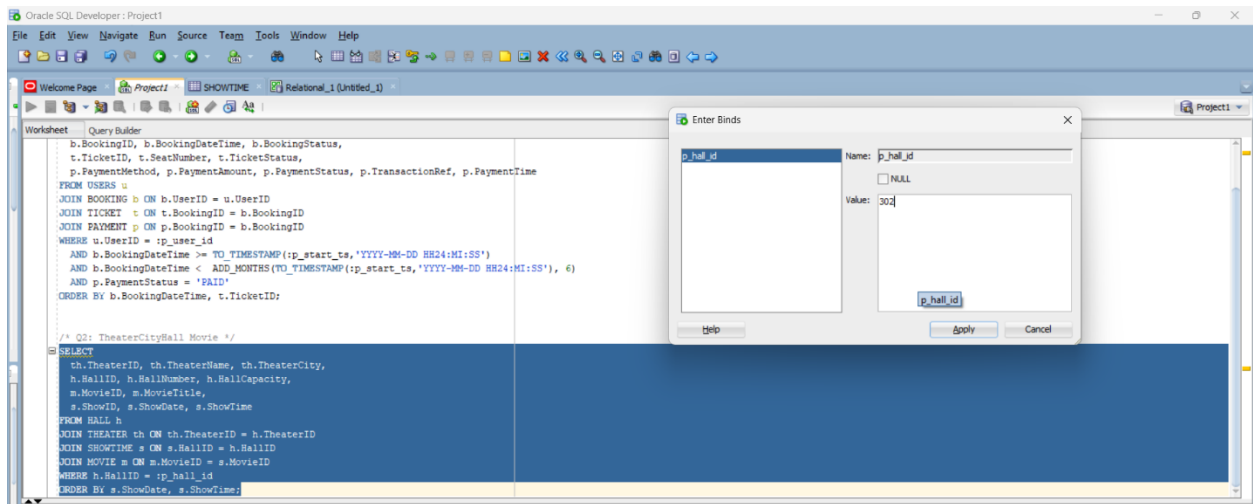


Figure 50: Assigning Hall id

Query Result x

All Rows Fetched: 2 in 0.057 seconds

	THEATERID	THEATERNAME	THEATERCITY	HALLID	HALLNUMBER	HALLCAPACITY	MOVIEID	MOVIEITITLE	SHOWID	SHOWDATE	SHOWTIME
1	202	Pokhara Cineplex Pokhara	Pokhara	302	1	180	101	Avatar: Fire and Ash	403	21-DEC-25	10:00
2	202	Pokhara Cineplex Pokhara	Pokhara	302	1	180	105	Dune: Part Two	404	21-DEC-25	18:00

Figure 51: Output of above query

7.2.20 Query 3

```

/* Q3: MovieTheaterCityHallOccupancyPerformer (Top 3) */
SELECT *
FROM (
  SELECT
    th.TheaterName,
    th.TheaterCity,
    h.HallNumber,
    h.HallCapacity,
    COUNT(DISTINCT s.ShowID) AS TotalShows,
    COUNT(t.TicketID)       AS PaidTickets,
    ROUND (
      (COUNT(t.TicketID) / NULLIF(h.HallCapacity * COUNT(DISTINCT s.ShowID), 0)) * 100,
      2
    ) AS OccupancyPercent
  FROM SHOWTIME s
  JOIN HALL h      ON h.HallID = s.HallID
  JOIN THEATER th  ON th.TheaterID = h.TheaterID
  JOIN BOOKING b   ON b.ShowID = s.ShowID
  JOIN PAYMENT p   ON p.BookingID = b.BookingID AND p.PaymentStatus = 'PAID'
  JOIN TICKET t    ON t.BookingID = b.BookingID AND t.TicketStatus = 'PAID'
  WHERE s.MovieID = :p_movie_id
  GROUP BY th.TheaterName, th.TheaterCity, h.HallNumber, h.HallCapacity
  ORDER BY OccupancyPercent DESC
)
WHERE ROWNUM <= 3;

```

Figure 52: Creating MovieTheaterCityHall Occupancy performer table

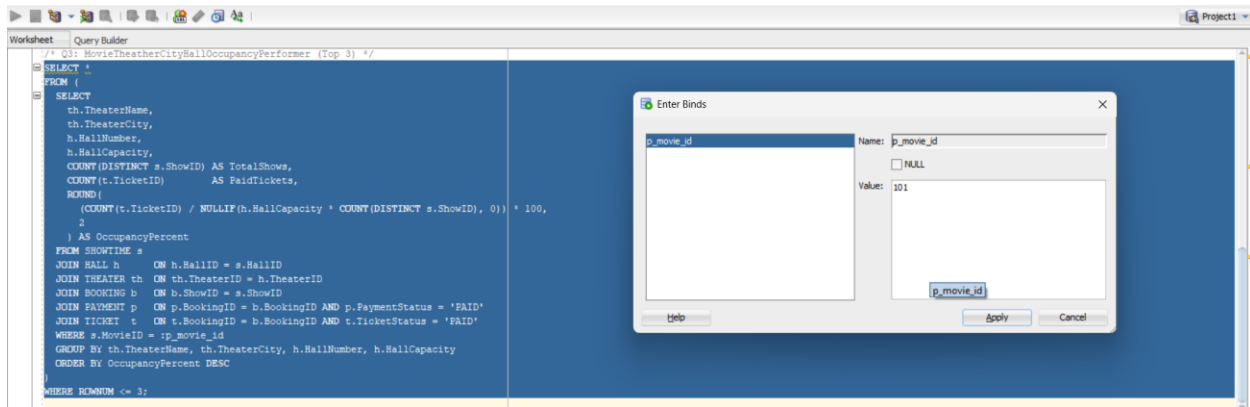


Figure 53: Assigning movie_ID

Query Result x

All Rows Fetched: 3 in 0.366 seconds

	THEATERNAME	THEATERCITY	HALLNUMBER	HALLCAPACITY	TOTALSHOWS	PAIDTICKETS	OCCUPANCYPERCENT
1	Kumari Cinemas	Kathmandu	1	200	1	3	1.5
2	Itahari City Cinema	Itahari	1	160	1	1	0.63
3	Pokhara Cineplex	Pokhara	1	180	1	1	0.56

Figure 54: Output of above query

8 Conclusion

This milestone successfully produced a structured and reliable database design for the KumariCinemas ticketing system by applying full normalisation from UNF to 3NF. The unnormalised dataset initially contained repeating groups for halls, show slots, pricing conditions, and ticket transactions, which created redundancy and risked inconsistencies during updates. Through First Normal Form, all attributes were made atomic and repeating groups were separated into distinct relations. Second Normal Form introduced stable identifiers such as UserID, TheaterID, HallID, ShowID, and PolicyID to eliminate partial dependency and remove reliance on name-based composite keys. Third Normal Form confirmed that no transitive dependency remained and ensured every non-key attribute depends only on its primary key. The final schema is consistent with the final ERD and supports key business rules such as automated cancellation tracking and variable pricing for public holidays and new release weeks. The database was implemented successfully in Oracle 19c using Data Modeler generated scripts and validated using SQL queries. Overall, the design is scalable, maintainable, and suitable for integration with the ASP.NET web application.

9 References

Oracle, 2025. *Oracle*. [Online]

Available at: <https://www.oracle.com/database/sqldeveloper/technologies/sql-data-modeler/>

Oracle, 2025. *Oracle*. [Online]

Available at: <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlug/>

Paradigm, V., 2025. *Visual Paradigm*. [Online]

Available at: <https://www.visual-paradigm.com/guide/data-modeling/what-is-entity-relationship-diagram/>

10 Appendix

1) CREATE TABLES (3NF)

```
CREATE TABLE USERS (  
    UserID    NUMBER(10) PRIMARY KEY,  
    Username  VARCHAR2(80) NOT NULL,  
    Address   VARCHAR2(80) NOT NULL  
);
```

```
CREATE TABLE MOVIE (  
    MovieID   NUMBER(10) PRIMARY KEY,  
    MovieTitle VARCHAR2(150) NOT NULL,  
    Duration  NUMBER(5),  
    Language  VARCHAR2(50) NOT NULL,  
    Genre     VARCHAR2(50) NOT NULL,  
    ReleaseDate DATE  
);
```

```
CREATE TABLE THEATER (  
    TheaterID NUMBER(10) PRIMARY KEY,  
    TheaterName VARCHAR2(120) NOT NULL,  
    TheaterCity VARCHAR2(80) NOT NULL  
);
```

```
CREATE TABLE HALL (  
    HallID    NUMBER(10) PRIMARY KEY,
```

```
TheaterID  NUMBER(10) NOT NULL,  
HallNumber  NUMBER(5) NOT NULL,  
HallCapacity  NUMBER(5) NOT NULL,  
CONSTRAINT FK_HALL_THEATER  
    FOREIGN KEY (TheaterID) REFERENCES THEATER(TheaterID),  
CONSTRAINT CK_HALL_CAPACITY  
    CHECK (HallCapacity > 0),  
CONSTRAINT UQ_THEATER_HALLNUMBER  
    UNIQUE (TheaterID, HallNumber)  
);
```

```
CREATE TABLE SHOWTIME (  
    ShowID  NUMBER(10) PRIMARY KEY,  
    HallID  NUMBER(10) NOT NULL,  
    MovieID  NUMBER(10) NOT NULL,  
    ShowDate  DATE NOT NULL,  
    ShowTime  VARCHAR2(8) NOT NULL, -- e.g. '10:00'  
    CONSTRAINT FK_SHOWTIME_HALL  
        FOREIGN KEY (HallID) REFERENCES HALL(HallID),  
    CONSTRAINT FK_SHOWTIME_MOVIE  
        FOREIGN KEY (MovieID) REFERENCES MOVIE(MovieID),  
    CONSTRAINT UQ_HALL_SHOWDATETIME  
        UNIQUE (HallID, ShowDate, ShowTime)  
);
```

```
CREATE TABLE BOOKING (  
    BookingID  NUMBER(10) PRIMARY KEY,
```

```
UserID      NUMBER(10)  NOT NULL,
ShowID      NUMBER(10)  NOT NULL,
BookingDateTime TIMESTAMP NOT NULL,
BookingStatus VARCHAR2(20) NOT NULL,
CONSTRAINT FK_BOOKING_USER
    FOREIGN KEY (UserID) REFERENCES USERS(UserID),
CONSTRAINT FK_BOOKING_SHOWTIME
    FOREIGN KEY (ShowID) REFERENCES SHOWTIME(ShowID),
CONSTRAINT CK_BOOKING_STATUS
    CHECK (BookingStatus IN ('BOOKED','CONFIRMED','CANCELLED','EXPIRED'))
);
```

```
CREATE TABLE PRICE_POLICY (
    PolicyID      NUMBER(10)  PRIMARY KEY,
    HallID        NUMBER(10)  NOT NULL,
    MovieID       NUMBER(10)  NOT NULL,
    EffectiveFromDate DATE     NOT NULL,
    EffectiveToDate DATE,
    IsPublicHoliday CHAR(1)    NOT NULL,
    IsNewReleaseWeek CHAR(1)    NOT NULL,
    TicketPrice   NUMBER(10,2) NOT NULL,
    CONSTRAINT FK_PP_HALL
        FOREIGN KEY (HallID) REFERENCES HALL(HallID),
    CONSTRAINT FK_PP_MOVIE
        FOREIGN KEY (MovieID) REFERENCES MOVIE(MovieID),
    CONSTRAINT CK_PP_FLAGS
        CHECK (IsPublicHoliday IN ('Y','N') AND IsNewReleaseWeek IN ('Y','N')),
```

```
CONSTRAINT CK_PP_PRICE
    CHECK (TicketPrice > 0)
);

CREATE TABLE TICKET (
    TicketID    NUMBER(10) PRIMARY KEY,
    BookingID   NUMBER(10) NOT NULL,
    SeatNumber  VARCHAR2(10) NOT NULL,
    TicketStatus VARCHAR2(20) NOT NULL,
    CONSTRAINT FK_TICKET_BOOKING
        FOREIGN KEY (BookingID) REFERENCES BOOKING(BookingID),
    CONSTRAINT CK_TICKET_STATUS
        CHECK (TicketStatus IN ('RESERVED','PAID','CANCELLED','REFUNDED')),
    CONSTRAINT UQ_BOOKING_SEAT
        UNIQUE (BookingID, SeatNumber)
);
```

```
CREATE TABLE PAYMENT (
    PaymentID   NUMBER(10) PRIMARY KEY,
    BookingID   NUMBER(10) NOT NULL,
    PaymentMethod VARCHAR2(20) NOT NULL,
    PaymentAmount NUMBER(10,2) NOT NULL,
    PaymentStatus VARCHAR2(20) NOT NULL,
    TransactionRef VARCHAR2(40),
    PaymentTime  TIMESTAMP,
    CONSTRAINT FK_PAYMENT_BOOKING
        FOREIGN KEY (BookingID) REFERENCES BOOKING(BookingID),
```

```
CONSTRAINT CK_PAYMENT_METHOD
CHECK (PaymentMethod IN ('CASH','CARD','ESEWA','STRIPE')),
CONSTRAINT CK_PAYMENT_STATUS
CHECK (PaymentStatus IN ('PAID','FAILED','PENDING','REFUNDED')),
CONSTRAINT CK_PAYMENT_AMOUNT
CHECK (PaymentAmount >= 0)
);
```

2) INSERT DATA

```
INSERT INTO USERS VALUES (1,'Aashish Khadka','Itahari');
```

```
INSERT INTO USERS VALUES (2,'Jeewan Rai','Pokhara');
```

```
INSERT INTO USERS VALUES (3,'Sita Karki','Biratnagar');
```

```
INSERT INTO USERS VALUES (4,'Nisha Giri','Lalitpur');
```

```
INSERT INTO USERS VALUES (5,'Prakash Karki','Itahari');
```

```
INSERT INTO MOVIE VALUES (101,'Avatar: Fire and Ash',165,'English','Fiction', DATE
'2025-12-19');
```

```
INSERT INTO MOVIE VALUES (102,'Salaar: Part 1 – Ceasefire',175,'Hindi','Action', DATE
'2023-12-22');
```

```
INSERT INTO MOVIE VALUES (103,'Interstellar',169,'English','Sci-Fi', DATE '2014-11-07');
```

```
INSERT INTO MOVIE VALUES (104,'Jatra',145,'Nepali','Comedy', DATE '2016-05-13');
```

```
INSERT INTO MOVIE VALUES (105,'Dune: Part Two',166,'English','Sci-Fi', DATE '2024-03-
01');
```

```
INSERT INTO THEATER VALUES (201,'Kumari Cinemas','Kathmandu');
```

```
INSERT INTO THEATER VALUES (202,'Pokhara Cineplex','Pokhara');
```

INSERT INTO THEATER VALUES (203,'Labim Mall Cinema','Lalitpur');

INSERT INTO THEATER VALUES (204,'Biratnagar Cineplex','Biratnagar');

INSERT INTO THEATER VALUES (205,'Itahari City Cinema','Itahari');

INSERT INTO HALL VALUES (301,201,1,200);

INSERT INTO HALL VALUES (302,202,1,180);

INSERT INTO HALL VALUES (303,203,2,183);

INSERT INTO HALL VALUES (304,204,1,220);

INSERT INTO HALL VALUES (305,205,1,160);

INSERT INTO SHOWTIME VALUES (401,301,101,DATE '2025-12-21','10:00');

INSERT INTO SHOWTIME VALUES (402,301,101,DATE '2025-12-21','14:00');

INSERT INTO SHOWTIME VALUES (403,302,101,DATE '2025-12-21','10:00');

INSERT INTO SHOWTIME VALUES (404,302,105,DATE '2025-12-21','18:00');

INSERT INTO SHOWTIME VALUES (405,303,101,DATE '2025-12-21','16:00');

INSERT INTO SHOWTIME VALUES (406,303,103,DATE '2025-12-21','20:00');

INSERT INTO SHOWTIME VALUES (407,304,101,DATE '2025-12-21','12:00');

INSERT INTO SHOWTIME VALUES (408,304,102,DATE '2025-12-21','18:00');

INSERT INTO SHOWTIME VALUES (409,305,104,DATE '2025-12-21','11:00');

INSERT INTO SHOWTIME VALUES (410,305,101,DATE '2025-12-21','19:00');

INSERT INTO PRICE_POLICY VALUES (501,301,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',450.00);

INSERT INTO PRICE_POLICY VALUES (502,301,101,DATE '2025-12-20',DATE '2025-12-22','Y','Y',550.00);

INSERT INTO PRICE_POLICY VALUES (503,302,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',420.00);

```
INSERT INTO PRICE_POLICY VALUES (504,302,105,DATE '2025-12-01',DATE '2025-12-31','N','N',480.00);
```

```
INSERT INTO PRICE_POLICY VALUES (505,303,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',430.00);
```

```
INSERT INTO PRICE_POLICY VALUES (506,303,103,DATE '2025-12-01',DATE '2025-12-31','N','N',400.00);
```

```
INSERT INTO PRICE_POLICY VALUES (507,304,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',460.00);
```

```
INSERT INTO PRICE_POLICY VALUES (508,304,102,DATE '2025-12-01',DATE '2025-12-31','N','N',440.00);
```

```
INSERT INTO PRICE_POLICY VALUES (509,305,104,DATE '2025-12-01',DATE '2025-12-31','N','N',350.00);
```

```
INSERT INTO PRICE_POLICY VALUES (510,305,101,DATE '2025-12-01',DATE '2025-12-31','N','Y',410.00);
```

```
INSERT INTO BOOKING VALUES (601,1,401,TO_TIMESTAMP('2025-12-20 10:15:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
```

```
INSERT INTO BOOKING VALUES (602,2,401,TO_TIMESTAMP('2025-12-20 11:10:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
```

```
INSERT INTO BOOKING VALUES (603,3,402,TO_TIMESTAMP('2025-12-20 12:05:00','YYYY-MM-DD HH24:MI:SS'),'BOOKED');
```

```
INSERT INTO BOOKING VALUES (604,4,403,TO_TIMESTAMP('2025-12-20 12:40:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
```

```
INSERT INTO BOOKING VALUES (605,5,404,TO_TIMESTAMP('2025-12-20 13:20:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
```

```
INSERT INTO BOOKING VALUES (606,1,405,TO_TIMESTAMP('2025-12-20 14:10:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
```

```
INSERT INTO BOOKING VALUES (607,2,407,TO_TIMESTAMP('2025-12-20 15:30:00','YYYY-MM-DD HH24:MI:SS'),'CANCELLED');
```

```
INSERT INTO BOOKING VALUES (608,3,408,TO_TIMESTAMP('2025-12-20 16:45:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
```

```
INSERT INTO BOOKING VALUES (609,4,410,TO_TIMESTAMP('2025-12-20
17:10:00','YYYY-MM-DD HH24:MI:SS'),'CONFIRMED');
```

```
INSERT INTO BOOKING VALUES (610,5,406,TO_TIMESTAMP('2025-12-20
18:05:00','YYYY-MM-DD HH24:MI:SS'),'BOOKED');
```

```
INSERT INTO TICKET VALUES (701,601,'A1','PAID');
```

```
INSERT INTO TICKET VALUES (702,601,'A2','PAID');
```

```
INSERT INTO TICKET VALUES (703,602,'B1','PAID');
```

```
INSERT INTO TICKET VALUES (704,603,'C1','RESERVED');
```

```
INSERT INTO TICKET VALUES (705,603,'C2','RESERVED');
```

```
INSERT INTO TICKET VALUES (706,604,'D5','PAID');
```

```
INSERT INTO TICKET VALUES (707,605,'E3','PAID');
```

```
INSERT INTO TICKET VALUES (708,605,'E4','PAID');
```

```
INSERT INTO TICKET VALUES (709,606,'F1','PAID');
```

```
INSERT INTO TICKET VALUES (710,607,'G2','CANCELLED');
```

```
INSERT INTO TICKET VALUES (711,608,'H1','PAID');
```

```
INSERT INTO TICKET VALUES (712,609,'J9','PAID');
```

```
INSERT INTO PAYMENT VALUES (801,601,'ESEWA',900.00,'PAID','ESW-601-
01',TO_TIMESTAMP('2025-12-20 10:20:00','YYYY-MM-DD HH24:MI:SS'));
```

```
INSERT INTO PAYMENT VALUES (802,602,'CARD',420.00,'PAID','CRD-602-
01',TO_TIMESTAMP('2025-12-20 11:15:00','YYYY-MM-DD HH24:MI:SS'));
```

```
INSERT INTO PAYMENT VALUES (803,603,'ESEWA',0.00,'PENDING','ESW-603-
01',NULL);
```

```
INSERT INTO PAYMENT VALUES (804,604,'CASH',420.00,'PAID','CSH-604-
01',TO_TIMESTAMP('2025-12-20 12:45:00','YYYY-MM-DD HH24:MI:SS'));
```

```
INSERT INTO PAYMENT VALUES (805,605,'STRIPE',960.00,'PAID','STP-605-
01',TO_TIMESTAMP('2025-12-20 13:25:00','YYYY-MM-DD HH24:MI:SS'));
```

```
INSERT INTO PAYMENT VALUES (806,606,'CARD',430.00,'PAID','CRD-606-
01',TO_TIMESTAMP('2025-12-20 14:15:00','YYYY-MM-DD HH24:MI:SS'));
```

```
INSERT INTO PAYMENT VALUES (807,607,'CARD',0.00,'REFUNDED','CRD-607-01',TO_TIMESTAMP('2025-12-20 15:35:00','YYYY-MM-DD HH24:MI:SS'));
```

```
INSERT INTO PAYMENT VALUES (808,608,'ESEWA',440.00,'PAID','ESW-608-01',TO_TIMESTAMP('2025-12-20 16:50:00','YYYY-MM-DD HH24:MI:SS'));
```

```
INSERT INTO PAYMENT VALUES (809,609,'CARD',410.00,'PAID','CRD-609-01',TO_TIMESTAMP('2025-12-20 17:15:00','YYYY-MM-DD HH24:MI:SS'));
```

```
INSERT INTO PAYMENT VALUES (810,610,'CASH',0.00,'FAILED','CSH-610-01',TO_TIMESTAMP('2025-12-20 18:10:00','YYYY-MM-DD HH24:MI:SS'));
```

```
COMMIT;
```

3) SELECT (print-out for all tables)

```
SELECT * FROM USERS ORDER BY UserID;
```

```
SELECT * FROM MOVIE ORDER BY MovieID;
```

```
SELECT * FROM THEATER ORDER BY TheaterID;
```

```
SELECT * FROM HALL ORDER BY HallID;
```

```
SELECT * FROM SHOWTIME ORDER BY ShowID;
```

```
SELECT * FROM BOOKING ORDER BY BookingID;
```

```
SELECT * FROM PRICE_POLICY ORDER BY PolicyID;
```

```
SELECT * FROM TICKET ORDER BY TicketID;
```

```
SELECT * FROM PAYMENT ORDER BY PaymentID;
```

4) REQUIRED COMPLEX QUERIES

```
/* Q1: User Ticket (6 months) */
```

```
SELECT
```

```
    u.UserID, u.Username, u.Address,
```

```
b.BookingID, b.BookingDateTime, b.BookingStatus,  
t.TicketID, t.SeatNumber, t.TicketStatus,  
p.PaymentMethod, p.PaymentAmount, p.PaymentStatus, p.TransactionRef, p.PaymentTime  
FROM USERS u  
JOIN BOOKING b ON b.UserID = u.UserID  
JOIN TICKET t ON t.BookingID = b.BookingID  
JOIN PAYMENT p ON p.BookingID = b.BookingID  
WHERE u.UserID = :p_user_id  
AND b.BookingDateTime >= TO_TIMESTAMP(:p_start_ts,'YYYY-MM-DD HH24:MI:SS')  
AND b.BookingDateTime < ADD_MONTHS(TO_TIMESTAMP(:p_start_ts,'YYYY-MM-DD  
HH24:MI:SS'), 6)  
AND p.PaymentStatus = 'PAID'  
ORDER BY b.BookingDateTime, t.TicketID;
```

/* Q2: TheaterCityHall Movie */

```
SELECT  
th.TheaterID, th.TheaterName, th.TheaterCity,  
h.HallID, h.HallNumber, h.HallCapacity,  
m.MovieID, m.MovieTitle,  
s.ShowID, s.ShowDate, s.ShowTime  
FROM HALL h  
JOIN THEATER th ON th.TheaterID = h.TheaterID  
JOIN SHOWTIME s ON s.HallID = h.HallID  
JOIN MOVIE m ON m.MovieID = s.MovieID  
WHERE h.HallID = :p_hall_id  
ORDER BY s.ShowDate, s.ShowTime;
```

```
/* Q3: MovieTheaterCityHallOccupancyPerformer (Top 3) */  
SELECT *  
FROM (  
    SELECT  
        th.TheaterName,  
        th.TheaterCity,  
        h.HallNumber,  
        h.HallCapacity,  
        COUNT(DISTINCT s.ShowID) AS TotalShows,  
        COUNT(t.TicketID) AS PaidTickets,  
        ROUND(  
            (COUNT(t.TicketID) / NULLIF(h.HallCapacity * COUNT(DISTINCT s.ShowID), 0)) *  
            100,  
            2  
        ) AS OccupancyPercent  
    FROM SHOWTIME s  
    JOIN HALL h ON h.HallID = s.HallID  
    JOIN THEATER th ON th.TheaterID = h.TheaterID  
    JOIN BOOKING b ON b.ShowID = s.ShowID  
    JOIN PAYMENT p ON p.BookingID = b.BookingID AND p.PaymentStatus = 'PAID'  
    JOIN TICKET t ON t.BookingID = b.BookingID AND t.TicketStatus = 'PAID'  
    WHERE s.MovieID = :p_movie_id  
    GROUP BY th.TheaterName, th.TheaterCity, h.HallNumber, h.HallCapacity  
    ORDER BY OccupancyPercent DESC  
)  
WHERE ROWNUM <= 3;
```

